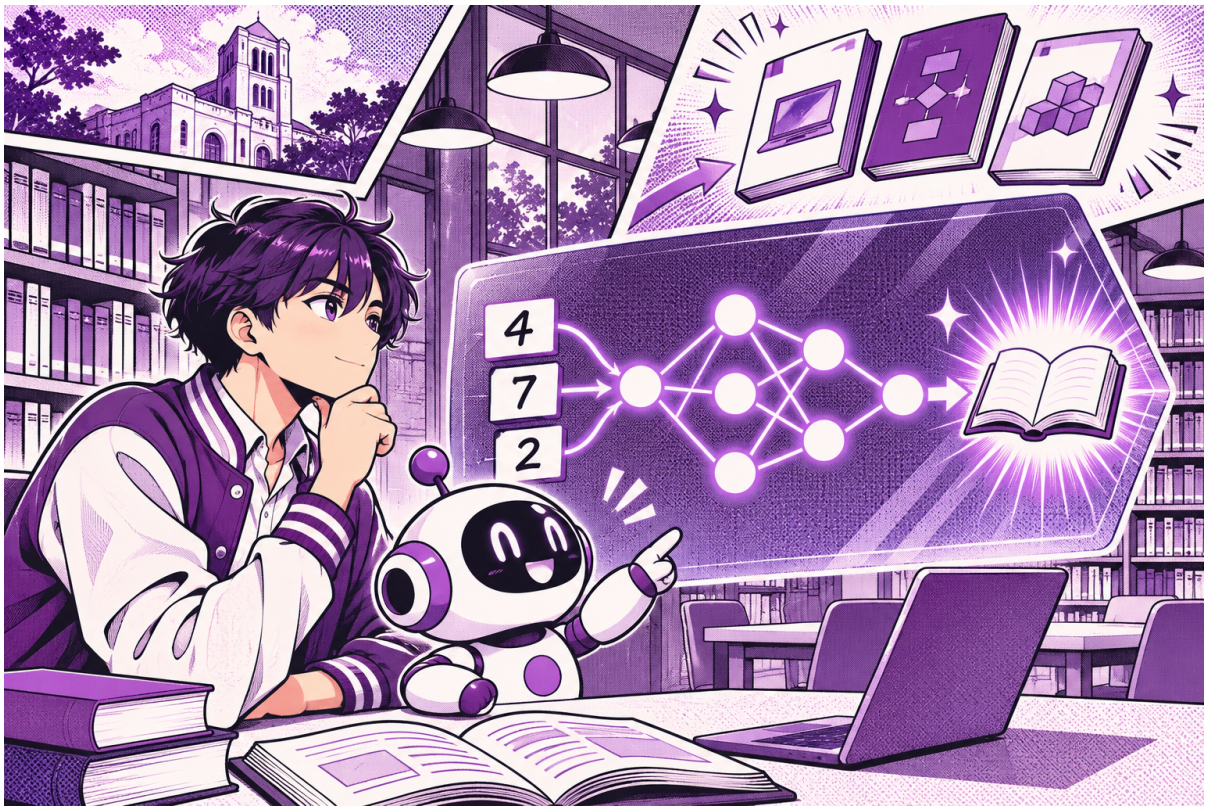
ILLUSTRATED LEARNING BOOK | VOLUME 3

Computing Made Understandable

Connectionist AI



Original manga illustration: a story analogy. Formal labeled diagrams inside the book carry the technical definitions.

Build a neural prediction one calculation at a time. Learn how parameters change, how evidence is evaluated and how people remain part of the design.

First-year learning edition | 6 November 2026

K.I.V.A.S. Learning Lab | Foundations of Computer Science & Artificial Intelligence

Inside this book

Chapters are separate and numbered within each volume. Printed page numbers match this PDF. The PDF outline also includes teaching sections.

How to learn from this volume 3

What Does It Mean for a Machine to Learn? 4

From Biological Inspiration to Artificial Neurons 9

Features, Weights and Bias 14

Activation Functions 20

Building a Neural Network 25

Forward Propagation 29

How Neural Networks Learn 35

Evaluating a Model and Generalization 42

Neural Networks in Everyday Applications 48

Limitations, Ethics and Responsible AI 53

Smart Campus Library: integrated project 59

Glossary and retrieval cues 65

Reading, sources and credits 67

How to learn from this volume

You do not need to know a programming language. Begin with the story, predict a result, then trace the formal example. Cover the feedback until you have written a reason. A diagram is useful when you can explain what each box, arrow and number means.

Learning move	What to do	Evidence to keep
Predict	Write the next state before reading it	Your first answer
Trace	Apply one operation at a time	Intermediate states
Explain	Name the rule that justifies the change	A sentence or labeled diagram
Transfer	Change one input or boundary case	A new trace
Retrieve	Return later without the solution	A corrected explanation

Conventions used throughout the set

Indices start at zero. Arrays use length for current entries and capacity for reserved slots. Binary search uses inclusive low and high bounds and the lower midpoint. A missing result is distinct from index zero. Tree height counts edges. Campus graph edges are two-way unless explicitly stated. ReLU(z)=max(0, z); the step rule includes zero when written z> =0. A raw neural score has only the meaning the task gives it. Illustrative data are not reported experiments.

Your route

Basic multiplication, addition and simple variables are enough to begin. This volume moves from data and individual units to networks, training, evaluation and responsible use. The final Smart Library project connects all three volumes.

A USEFUL STUDY RHYTHM

Try one or two sections in a 25-40 minute session. Finish with a small new example. After a day, repeat the chapter lab without looking at its solution. The separate Test Bank supplies unfamiliar cases, and the Learning Materials companion gives optional visual and interactive routes.

Artwork follows the supplied purple, black, white and violet theme. Manga scenes are narrative analogies. Labeled technical figures and worked tables define the algorithms and numerical examples. Essential information is also stated in text.

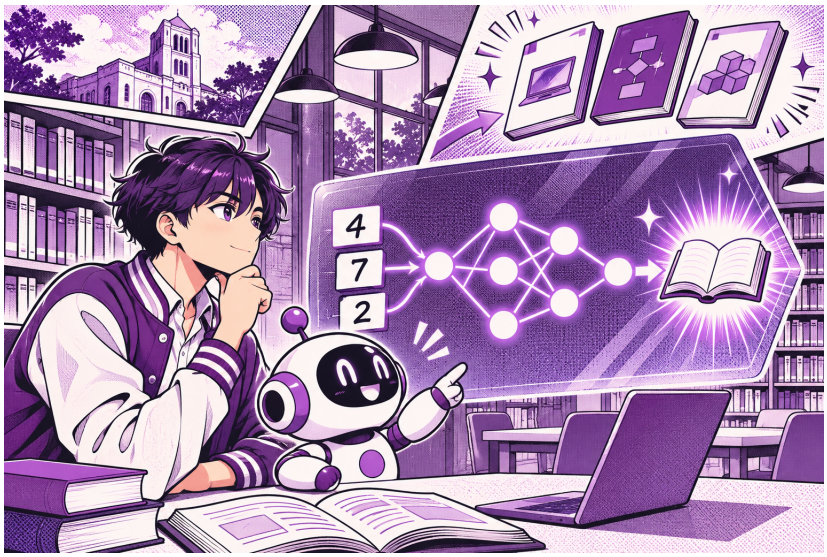
AI-CH01 / CHAPTER 01

What Does It Mean for a Machine to Learn?

BY THE END

Distinguish fixed rules, training, inference and learning signals.

Learning describes how a rule is obtained. A prediction still needs evidence before anyone relies on it.



ART-AI-CH01 The glowing network is a conceptual image of numerical inputs becoming a suggestion. Its decorative numbers are not the parameters of the worked network.

1.1 Can a machine recognize a preference?

“Where is book 24?” has an answer on a shelf slip. “Would I like book 24?” calls for an estimate. A clerk might listen to many readers describe books they enjoyed and notice recurring patterns. A learning system takes a numerical version of that approach.

In connectionist AI, linked computational units transform inputs into outputs using adjustable numbers. A training procedure chooses those numbers using examples. A trained model then applies them to new cases. The biological analogy behind the term neuron is loose. You can reproduce this chapter's entire artificial neuron with multiplication, addition and a rule for its output.

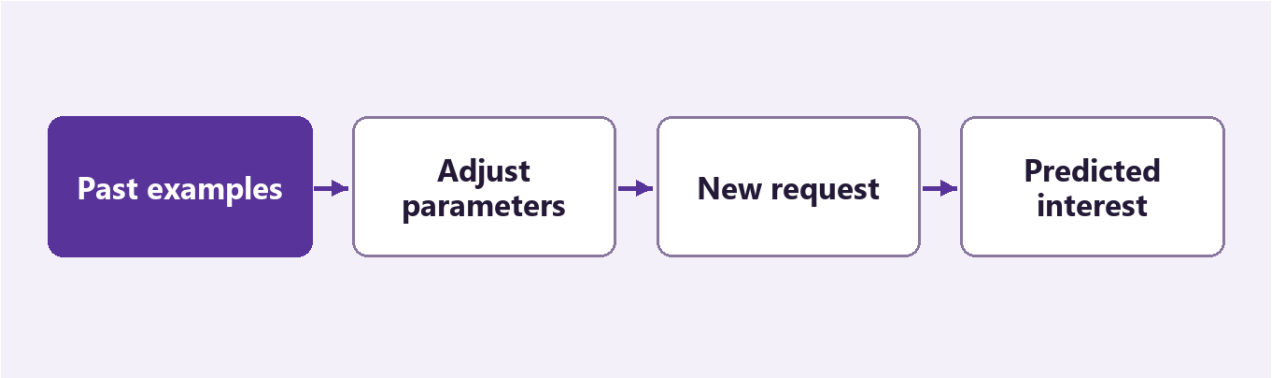


FIG-AI-01-01 During training examples influence stored parameters. Later a new request passes through the resulting model. The prediction is an estimate and a reader may disagree with it.

Look closer: A recommendation can be rejected without being a faulty arithmetic calculation. Record the input, model version and predicted score to distinguish a computational mistake from a disagreement with the learned pattern.

Separate a fact from a guess

Classify each of the following library jobs as a direct lookup, an explicit rule, or a possible learned prediction. State what evidence would let a clerk check the result.

Job	Method	Evidence
Find shelf for ID 24	Lookup	Catalogue record
Decline a closed path	Rule	Current closure flag
Estimate interest in a title	Prediction	Later voluntary feedback

Reading the result: A fixed rule can also produce recommendations, for instance by choosing a book with the requested subject. Learning is distinguished by how the model's parameters were obtained. A complicated rule written entirely by a person has not thereby learned from examples.

Try it yourself: If a clerk manually sets every weight in a toy neuron, what can its output demonstrate?

Check your reasoning: It demonstrates how that unit calculates. It provides no evidence that training found the weights or that the unit predicts real preferences accurately.

Another case: A table of handwritten suggestions

A librarian writes “offer a map book whenever a visitor asks for travel.” The computer follows that rule perfectly. Another system adjusts numerical parameters from reader feedback. Which one has learned in the sense used in this course?

Feedback: The second has adjusted its mapping from examples. The first automates a human-written decision. Both can be useful, and either can make bad suggestions. To tell how a recommendation was produced, inspect the source of its rule or parameters rather than judging from a single output.

1.2 Learning is a way of obtaining a rule

Suppose the librarian recommends every book with the tag astronomy. That is an explicit rule. Another program changes numerical parameters after comparing predictions with readers' voluntary ratings. That is learning from examples. Both programs can output recommendations; the difference lies in how the decision rule was obtained.

A model is a rule for producing an output from inputs. A parameter is a stored number that controls that rule. Training uses examples and a learning procedure to choose parameters. Inference uses the chosen parameters to make a prediction. A dataset is a collection of examples. The word learn does not imply that a machine understands books as a person does; it describes a measurable adjustment procedure and the behavior it produces on later cases.

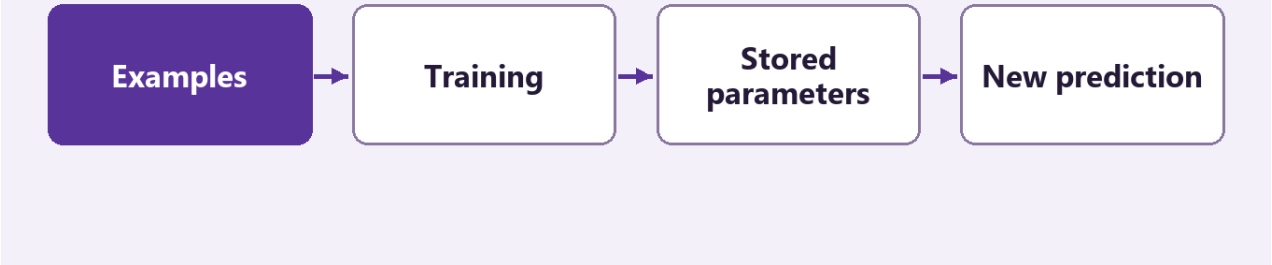


FIG-AI-01-02 Only the training stage changes the parameters in this simplified workflow. New examples can pass through the trained rule without changing it.

Investigate and explain

Classify the four jobs below. For each learned job, name the evidence that would be needed to judge it. Avoid treating a program's confidence or complexity as proof that it learned.

Job	How rule is obtained	Appropriate check
Match exact ID	Programmed equality	Compare requested and stored ID
Recommend same tag	Human-written rule	Check tag match
Predict voluntary rating	Fitted from examples	Compare later ratings on held-out cases
Re-run fixed model	Parameters unchanged	Reproduce inference arithmetic

Reading the result: Learning is useful when examples reveal a pattern that is hard to specify completely. It is unnecessary for facts already held in a trustworthy catalogue. A prediction can remain uncertain even when every multiplication is correct. Keep two questions separate: did the program calculate the rule correctly, and is that rule useful for the intended people and cases?

Try it yourself: If a toy network has hand-picked weights, has it demonstrated training?

Check your reasoning: No. It demonstrates a forward calculation. Evidence of training would include examples, a defined learning signal and a procedure that actually changes parameters.

1.3 Three kinds of learning signals

A teacher can show the correct answer, ask learners to find groups without supplied answers, or provide feedback after a sequence of actions. These everyday situations offer a first intuition for supervised, unsupervised and reinforcement learning. The analogy describes the feedback available; it does not make the machine a human pupil.

Supervised learning uses examples paired with target labels, such as an image and its digit. Unsupervised learning looks for structure without those task labels, such as grouping similar records. Reinforcement learning uses actions, an environment and a reward signal, often delayed over time. A reward is not automatically a complete description of what people value. This volume primarily teaches supervised neural networks, because small input-output calculations make the learning process easier to inspect.

FIG-AI-01-03 The boxes distinguish the signal used during learning. A single application can combine methods, so classify the actual learning procedure rather than its marketing label.

Investigate and explain

Match each classroom scenario to the signal it supplies. Explain why grouping books does not establish whether a reader will like a group, and why a robot's travel reward needs careful design.

Scenario	Signal	Family
Photos paired with digits	Correct class per example	Supervised
Group books by recorded features	No preference labels	Unsupervised
Robot tries routes and receives scores	Feedback after actions	Reinforcement

Reading the result: A cluster records similarity under chosen features, not a verified preference. A reward for speed alone might favor an unusable stairway or ignore safety unless the environment and constraints prevent it. Even an appropriate learning family needs good data and evaluation. Names describe the setup; they do not guarantee a useful result.

Try it yourself: Is a fixed rule that awards points already reinforcement learning?

Check your reasoning: No. There must also be a procedure that changes the action policy using the feedback. Calculating rewards alone does not demonstrate learning.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Propose one exact lookup, one explicit rule and one learned prediction for the library. For the prediction, name inputs, target and an evaluation comparison.

Worked guidance

Lookup can resolve copy 24; a rule can exclude closed paths; a prediction can estimate a voluntary later like from permitted features. Compare that prediction on suitable held-out cases against a simple subject-tag baseline. A manually set toy network does not establish learned usefulness.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can distinguish fixed rules, training, inference and learning signals. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH02 / CHAPTER 02

From Biological Inspiration to Artificial Neurons

BY THE END

Explain the biological analogy's limits and execute a neuron as an arithmetic procedure.

Connections provide an intuition. Numerical definitions provide the evidence for every calculation in this volume.

2.1 A useful analogy with clear limits

A biological brain contains connected cells. An artificial neural network also connects units that influence one another. That shared picture inspired a useful vocabulary, but the little unit in this book is an arithmetic rule, not a realistic brain cell. Pip's expressive face in the illustrations is a storytelling device, not evidence about machine feelings.

Biological neurons involve electrochemical processes, timing and complex interactions. Our artificial neuron takes numerical inputs, multiplies each by a weight, adds a bias and applies an activation function. A function is simply a rule that maps an input to an output. We use the analogy to introduce connections and adjustable influence, then use the mathematical definition to predict behavior. No claim about consciousness follows from this arithmetic model.

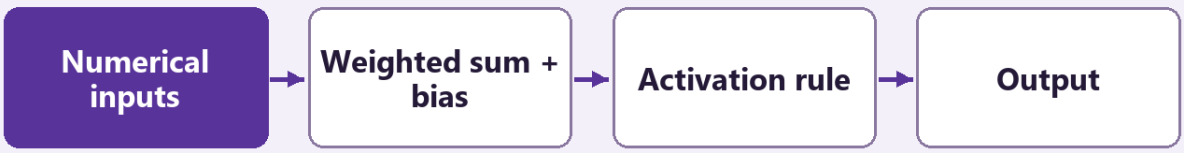


FIG-AI-02-01 Read left to right. The boxes are stages of a calculation, not anatomical parts. A number leaving one unit can become an input to another.

Investigate and explain

Explain the unit to a classmate using only the operations multiply, add and apply a rule. Use inputs 2 and 3, weights 0.5 and 0.2, bias 0.1 and an identity activation, which returns its input unchanged.

Stage	Calculation	Result
First contribution	2 x 0.5	1.0
Second contribution	3 x 0.2	0.6
Weighted sum plus bias	1.0 + 0.6 + 0.1	1.7
Identity activation	Return the sum	1.7

Reading the result: The output 1.7 is fully explained by the supplied arithmetic. It is neither a probability nor an intelligence score without an additional definition. The two weights pair with two inputs, while the bias is added once for the unit. Naming each stage is a reliable way to find an error before networks add more connections.

Try it yourself: Does the term neuron imply that this unit accurately simulates a brain cell?

Check your reasoning: No. It is a simplified computational abstraction inspired by connections. Use its stated equation to reason about it, and keep biological claims separate.

2.2 Many small units, one distributed calculation

Imagine several clerks each checking one aspect of a book request. One looks at subject overlap, another at author history. A later clerk combines their reports. This offers an intuition for a layered network: later values depend on earlier computed values. Unlike real clerks, the units do not decide their own purposes or understand their reports.

Connectionist means that the pattern of connections and their numerical parameters jointly shape the calculation. A hidden unit is a computed unit between inputs and outputs. Its value is called an activation after its activation function is applied. A diagram shows possible dependencies; it does not automatically tell us what concept each unit represents. Assigning a label such as likes science requires evidence, not simply an appealing story.

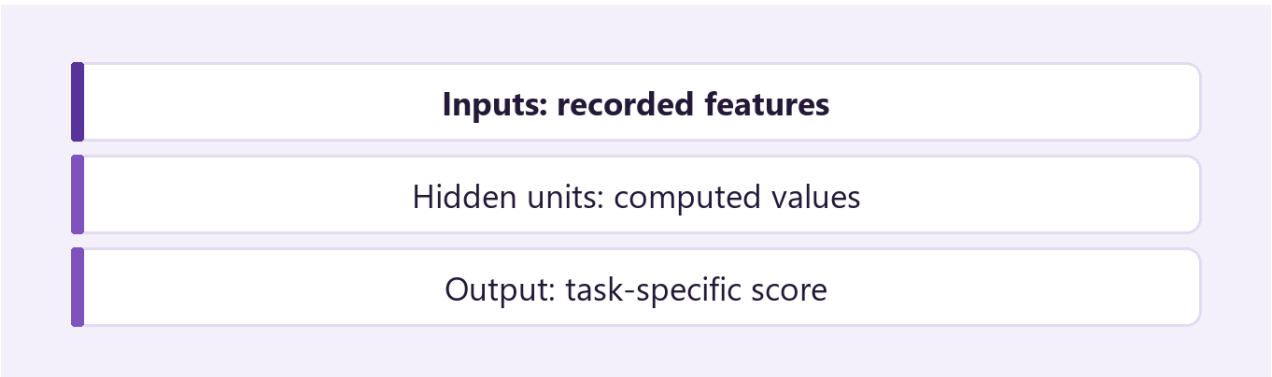


FIG-AI-02-02 A hidden value is produced by a rule. Its interpretation must come from analysis of the model and data rather than the location of its circle.

Investigate and explain

A hidden unit outputs max(0, x1 - x2). Evaluate three cases. Describe exactly what changes its value, then judge whether the nickname science lover is justified by these calculations.

x1	x2	Sum x1 - x2	Hidden output
2	1	1	1
1	2	-1	0
3	1	2	2

Reading the result: The unit responds to the positive part of the difference between its two features. Without knowing what those features mean and how examples are distributed, the nickname adds an unsupported claim. Even with meaningful features, a single unit may contribute to several patterns. For beginners, exact arithmetic is a safer explanation than pretending every hidden unit is a human concept detector.

Try it yourself: If two hidden units are drawn side by side, must one read the other?

Check your reasoning: No. Read the connecting arrows. In a standard feedforward layer, both can read the previous layer independently.

2.3 A neuron as a reusable procedure

Before assembling a network, turn the single unit into a small procedure that another learner can execute. Store inputs in one row and weights in a matching row. The alignment matters: a weight belongs to a particular input feature. Swapping only the feature order changes the rule even when both rows contain the same numbers.

For each pair, multiply input by its weight and add the result to a running total. Begin that total with the unit's bias, or begin with zero and add the bias after the loop. Either convention adds bias exactly once. Then apply the activation. This procedure works with two inputs or many inputs. Its cost grows with the number of incoming connections when basic arithmetic has constant cost.

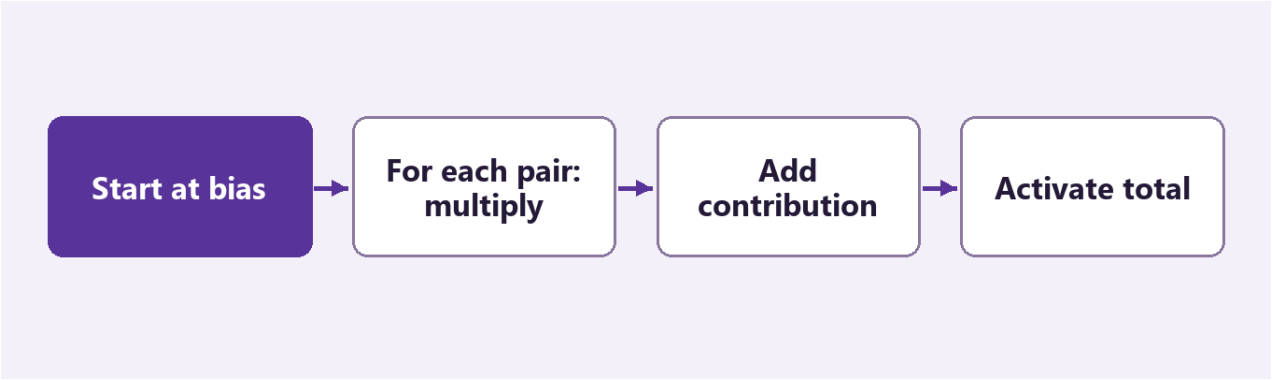


FIG-AI-02-03 The middle two steps repeat for every input-weight pair. The activation runs once after all contributions have been combined.

Investigate and explain

Use inputs [2, 3], weights [0.5, 0.2], bias 0.1. Trace a running sum initialized at bias. Then deliberately swap just the inputs and explain the difference.

Visit	Original total	Swapped-input total
Start	0.1	0.1
Pair 1	0.1 + 2 x 0.5 = 1.1	0.1 + 3 x 0.5 = 1.6
Pair 2	1.1 + 3 x 0.2 = 1.7	1.6 + 2 x 0.2 = 2.0

Reading the result: The mismatch is a data-contract problem rather than a multiplication error. A deployed model expects features in the order and units used during training. Consistency is part of the model. A copied weight table without feature definitions is insufficient to reproduce the intended rule.

Try it yourself: What if both input order and weight order are swapped together?

Check your reasoning: The paired products are unchanged, so their sum and the output are unchanged apart from possible floating-point rounding in larger implementations.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Explain an artificial neuron to a beginner and calculate inputs 2, 3 with weights 0.5, 0.2 and bias 0.1. Give one useful aspect and one limitation of the brain analogy.

Worked guidance

Multiply paired inputs and weights, add the single bias, then activate. The sum is 1.7. Connected influences offer a useful intuition; the arithmetic unit does not realistically reproduce the complex electrochemical behavior of a biological neuron.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can explain the biological analogy's limits and execute a neuron as an arithmetic procedure. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH03 / CHAPTER 03

Features, Weights and Bias

BY THE END

Define reproducible features, preserve input-weight pairing and separate contributions from bias.

Before trusting a calculation, make sure the numbers describe the intended observations in consistent units.

3.1 Turn observations into numbers

A machine needs a usable description of a case. A feature is one recorded property used as input to a model. For a reading request we might count shared subject tags or count earlier positive ratings for the same author with the reader's permission. Two clerks should get the same number from the same permitted records.

A label is the outcome the model is supposed to predict in supervised learning. A reader's later yes-or-no response can be a label. Using that later response itself as an input would give away the answer during training and leave a missing feature when a real request arrives.

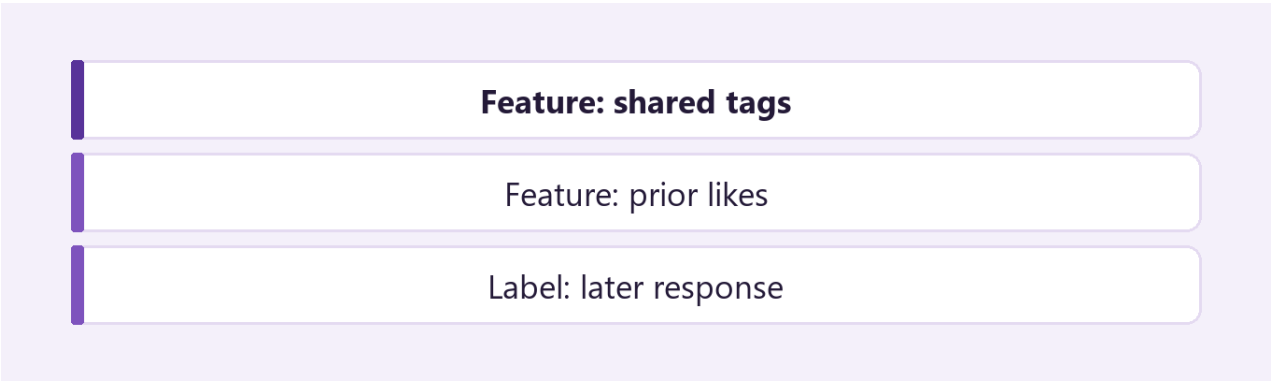


FIG-AI-03-01 The first two entries can be observed before the offer. The last arrives later and is useful for evaluating the offer. Timing is part of a feature definition.

Look closer: Suppose the labels come only from people who click a feedback form. A model trained on those responses may describe respondents better than quiet readers. The way examples enter the dataset helps determine what can be learned.

Design a feature a second clerk can reproduce

A visitor chooses tags maps, rivers, and travel. Candidate A has rivers and maps. Candidate B has travel and poetry. The visitor later likes B and skips A. Compute the shared-tag feature and distinguish each observed label from an unobserved preference.

Candidate	Shared tags	Shared count	Observation
A	Maps, rivers	2	Skipped
B	Travel	1	Liked
C, never shown	Unknown from brief	Unknown	No feedback

Reading the result: A skip can mean disinterest, lack of time, or a misplaced tap. Decide carefully whether it deserves a negative training label. A candidate never shown provides no observed response. Recording every unknown as dislike would distort the training examples.

Try it yourself: Would adding the number of letters in the reader's name necessarily improve predictions?

Check your reasoning: No. A measurable value may be irrelevant, intrusive or spuriously correlated. Select features that are justified, available at prediction time and appropriate for the task.

Another case: An answer smuggled into the features

A developer includes “clicked the suggested book tomorrow” as an input when training a recommendation model. It scores very highly on historical examples. At what exact point does the proposed system become unusable?

Feedback: When the desk must choose a suggestion today, tomorrow's click has not occurred. Historical evaluation used information unavailable at prediction time. Remove the future-derived input and re-evaluate on a split that respects the timing of requests and outcomes.

3.2 How strongly should each input count?

Suppose a toy interest score combines two quantities. The first input is the number of shared tags; the second is the number of earlier likes for the author. A weight tells us how much one unit of a feature contributes to the score while the other quantities stay fixed.

Multiplying a feature by its weight gives one contribution. A positive weight raises the sum as that input increases, a negative weight lowers it, and zero removes that direct contribution. The units matter: converting minutes to seconds changes the feature scale and generally requires a corresponding weight adjustment.

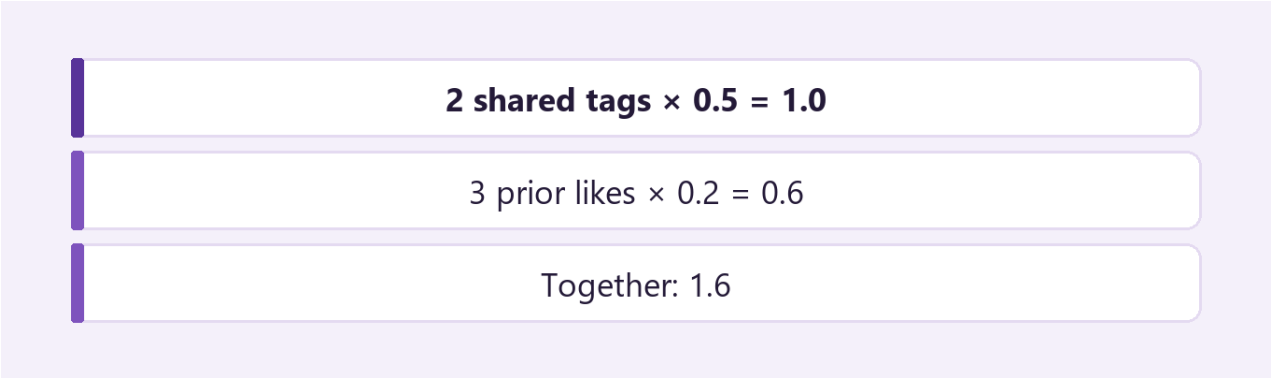


FIG-AI-03-02 Each product pairs one input with its own weight. Interchanging the weights changes the interpretation and can change the score even with the same two input values.

Look closer: Changing a weight by 0.1 changes its contribution by the input times 0.1. With input 3, that change is 0.3. The same parameter adjustment can affect different examples by different amounts.

Change one weight while holding inputs fixed

Start with inputs 2 and 3 and ignore any constant offset for the moment. Predict the score when only the second weight changes. Write both contributions before adding so you can see which changed.

Second weight	First contribution	Second contribution	Sum
0.2	1.0	0.6	1.6
0	1.0	0	1.0
-0.2	1.0	-0.6	0.4

Reading the result: A negative weight is meaningful in arithmetic without implying that a human trait is good or bad. Here it simply reverses the direction of a contribution. Whether any of these settings suits real users requires appropriate evidence.

Try it yourself: If the second input is zero, can changing only its weight change this sum?

Check your reasoning: No. Zero times any finite weight is zero. The weight still matters for another request whose second feature is nonzero.

Another case: Rescale a feature

A waiting-time feature is 2 minutes and its weight is 0.6 per minute. Convert the input to 120 seconds. Find a weight per second that preserves the same contribution, and describe the effect of forgetting to rescale it.

Feedback: The old contribution is 1.2. A weight of 0.01 per second gives 120 × 0.01 = 1.2. Keeping 0.6 after changing units would contribute 72. Feature units and weights must be documented together to reproduce a calculation.

3.3 The constant that shifts the score

A lamp controlled by daylight may need to turn on earlier in one room than another. Changing a baseline can move the decision point without changing how strongly light counts. In a neuron, bias is a constant added to the sum of weighted inputs. It is a numerical parameter, distinct from the societal biases we discuss in Volume III, Chapters 7 to 10.

Use x for an input, w for its matching weight, b for bias, and z for the total before activation. A small subscript identifies the first or second input. In words, multiply each input by its weight, add the products, then add the bias once.

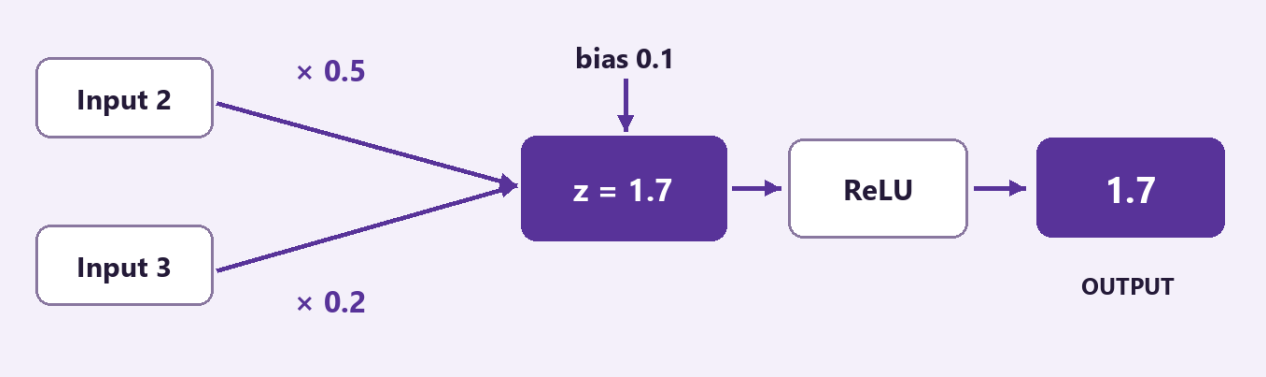


FIG-AI-03-03 The two branches contribute 1.0 and 0.6. The separate bias adds 0.1 at the sum, giving z = 1.7. The ReLU output rule leaves a positive total unchanged.

Look closer: If every input is zero, the sum before activation equals the bias. That quick check reveals a missing or double-counted bias without requiring any multiplication beyond zero.

Rebuild the team's neuron calculation

Copy the team's two-input example. Cover the intermediate total, calculate it, then repeat with a bias of -2.0. Apply the ReLU rule “return the larger of zero and z” only after building the entire sum.

Bias	Products	z	ReLU output
0.1	1.0 and 0.6	1.7	1.7
0	1.0 and 0.6	1.6	1.6
-2.0	1.0 and 0.6	-0.4	0

Reading the result: The last row changes the score and crosses the point where ReLU changes behaviour. If your first result was 1.8, check whether you added the 0.1 bias once on each branch. This neuron has one bias shared by its final sum.

Try it yourself: What is the output for inputs 1 and 2, weights 0.4 and 0.3, and bias 0.2?

Check your reasoning: The contributions are 0.4 and 0.6. Their sum plus 0.2 is 1.2. ReLU keeps that positive value.

Another case: Bias on an empty signal

Use inputs 0 and 0, arbitrary finite weights, bias -0.4 and ReLU. Then change only the bias to +0.4. Predict both scores and outputs. Which quantities did you never need to inspect?

Feedback: Both products vanish, so the first pre-activation score is -0.4 and the output zero. The second score and output are both 0.4. The weights do not affect these two cases because both inputs are zero, although they still belong to the model.

3.4 Units and categories are part of the input

The same waiting time can be recorded as two minutes or 120 seconds. A person sees the equivalence, but a model multiplies the number it receives. Keeping the same weight after a unit change alters the contribution. Feature names therefore need definitions, units, order and a rule for missing values.

A numerical feature represents a quantity, such as minutes. A category such as shelf zone A, B or C does not necessarily have a numerical distance. Coding the zones as 1, 2 and 3 can accidentally suggest that C is three times A. One-hot encoding is an optional simple alternative: create one indicator per category, with 1 for the present category and 0 for the others. A missing measurement is different from a genuine zero and requires a stated handling rule.

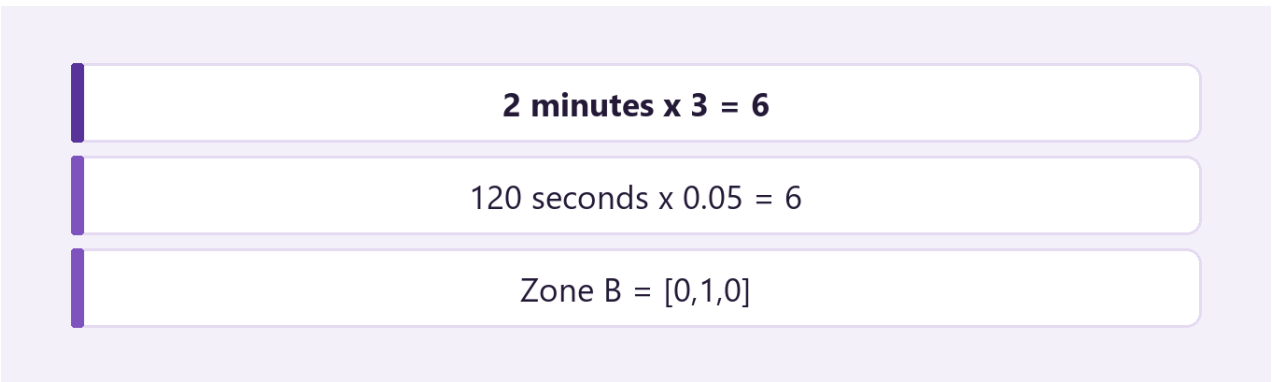


FIG-AI-03-04 The first two boxes preserve one contribution by converting the weight with the units. The third describes category membership, not a size or rank.

Investigate and explain

A feature in minutes has weight 3. Convert the input to seconds without changing its contribution. Then encode zone C using indicator order [A, B, C]. State how an unknown zone differs from a known A.

Case	Input	Weight or encoding	Meaning
Two minutes	2	3	Contribution 6
Same time in seconds	120	0.05	Contribution 6
Zone C	C	[0, 0, 1]	C present
Unknown zone	Missing	Policy required	Do not pretend it is A

Reading the result: Divide the old weight by 60 because the new numerical input is 60 times as large. This keeps this one linear contribution unchanged. Real preprocessing must be fitted using appropriate training data and repeated consistently at inference. The goal here is to recognize that data preparation is part of a complete model, not harmless formatting.

Try it yourself: Does converting categories to numbers prove that their differences are meaningful?

Check your reasoning: No. The encoding should reflect justified relationships. A numeric ID can be a name rather than a quantity.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

For x=[2, 3], w=[0.5, 0.2], b=0.1, compute z. Change only b to -2, then change only the first input of the original case to 4. Explain which terms changed.

Worked guidance

Original z=1.7. With b=-2, z=-0.4. With x1=4 and original bias, z=2+0.6+0.1=2.7. Bias shifts the sum independently of input values; changing a feature affects its paired weighted contribution.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can define reproducible features, preserve input-weight pairing and separate contributions from bias. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH04 / CHAPTER 04

Activation Functions

BY THE END

Apply step, ReLU and sigmoid conventions and explain why different sums can share an output.

The weighted sum is an intermediate result. Activation defines what the unit passes on.

4.1 Make a yes-or-no decision

An activation function is a repeatable rule that maps the sum z to an output. A step activation makes a hard decision. In this book define step(z) as 1 if z ≥ 0 and 0 otherwise. Writing down the equality convention avoids disagreement at the boundary.

One simple rule for two binary inputs is to count how many are 1 and subtract 1.5. With a step activation this unit says yes only if both inputs are present. The bias positions the dividing point between one and two positive inputs.

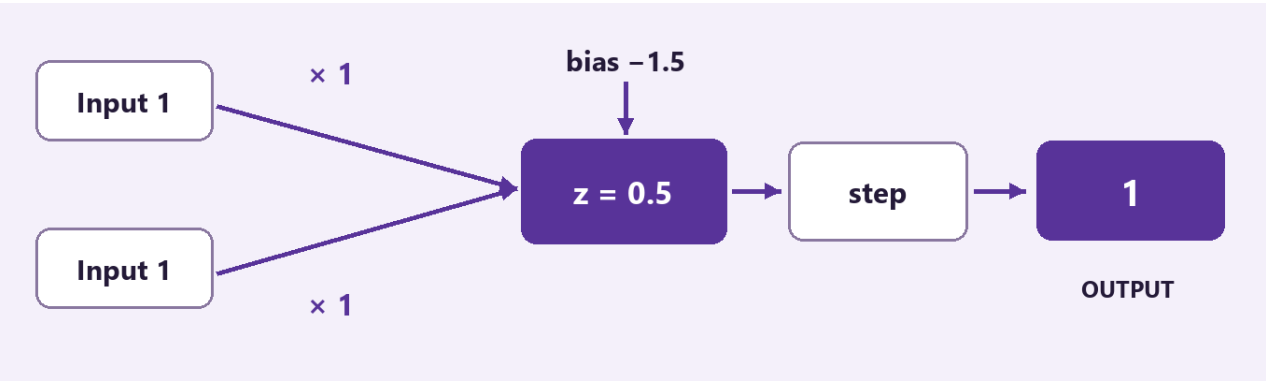


FIG-AI-04-01 Both incoming features contribute one. Subtracting 1.5 leaves a positive total, so the output is 1. The step output is a decision, not the intermediate score.

Look closer: A threshold rule can change class sharply over a tiny score change. Under the stated convention, -0.001 gives zero while +0.001 gives one. Report the underlying scores when diagnosing borderline decisions.

Test a threshold at its boundary

Use cards 0 and 1 to enumerate all four input pairs. Set both weights to one. Calculate z and step(z) for each pair. Then predict the effect of raising the bias to -0.5.

Inputs	z with -1.5	Step output
0, 0	-1.5	0
1, 0	-0.5	0
0, 1	-0.5	0
1, 1	0.5	1

Reading the result: With bias -0.5, either single positive input is sufficient and both positive inputs still give one. One change in a constant turns an "all present" rule into an "at least one present" rule. The output rule itself is identical in both trials.

Try it yourself: What happens if inputs sum to 1 and the bias is exactly -1?

Check your reasoning: z is zero. Our explicitly stated step convention returns one. A strict positive threshold would give a different answer on this boundary.

Another case: Design a gate from a sentence

Binary sensors report whether a visitor has a booking and a card. The gate should open if at least one sensor is 1. Try weights 1 and 1 with bias -0.5. Test all four pairs and decide whether the rule matches the sentence.

Feedback: The sums for 00, 01, 10, 11 are -0.5, 0.5, 0.5, 1.5. Step produces 0, 1, 1, 1. The check covers all four possible inputs in this tiny domain, unlike a test on just one visitor.

4.2 Three output rules for the same sum

Picture a volume knob with three possible behaviours. One snaps off or on, one stays at zero until the input turns positive and then follows it, and one changes smoothly within a limited range. The analogy helps distinguish step, ReLU and sigmoid.

ReLU returns max(0, z). Sigmoid smoothly maps a real-valued sum into the interval between zero and one. Its output is 0.5 at z = 0 and moves closer to the ends as z becomes very negative or positive. A number in this range requires an appropriate model and evaluation before it deserves a probability interpretation.

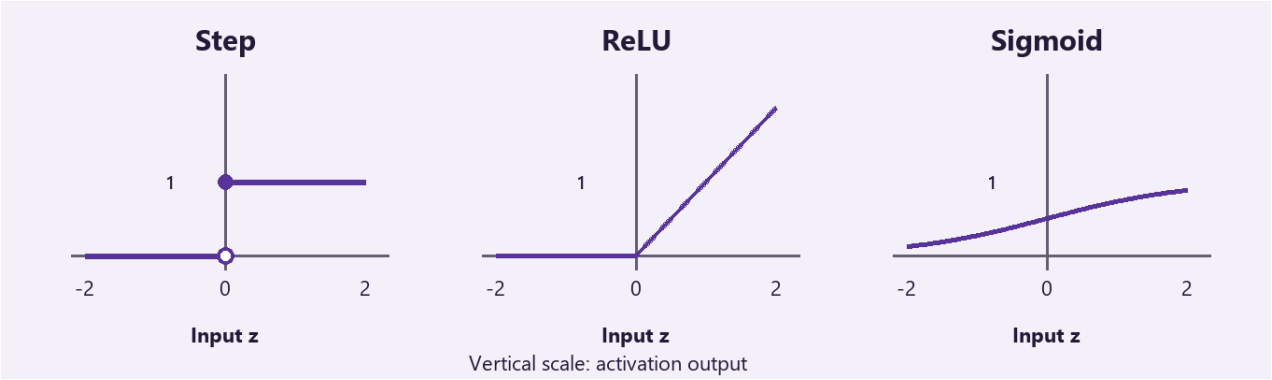


FIG-AI-04-02 Compare the three rules along the same horizontal input axis. The vertical axis measures activation output. The open and filled dots at zero distinguish the step's two branches.

Look closer: Compare a ReLU output of 2 with a sigmoid output near 0.88 at the same input. They belong to different output scales. Treat the rule and its units as part of the specification for any downstream unit.

Read an activation as a function

Use the sample inputs -2, 0 and 2 below. Round sigmoid values to two decimal places. Identify which rule preserves the size of positive scores and which provides only two distinct outputs.

z	Step	ReLU	Sigmoid approx.
-2	0	0	0.12
0	1	0	0.50
2	1	2	0.88

Reading the result: Step treats z = 2 and z = 20 alike. ReLU distinguishes them by returning 2 and 20. Sigmoid also distinguishes them mathematically, but both outputs lie below 1 and large positive inputs may look identical after rounding.

Try it yourself: If a unit outputs 0.88, can we conclude that 88% of readers will like this book?

Check your reasoning: The number alone is insufficient. That interpretation needs suitable labels, a probability-oriented model and checks on comparable held-out examples.

Another case: Recovering an input from its output

A ReLU unit reports zero. A second unit using our step rule also reports zero. What can you infer about each pre-activation sum? Does either output reveal the exact sum?

Feedback: For ReLU, the sum could be zero or any negative number. For step, a zero output means the sum was strictly negative, because equality returns one. Neither output tells you the exact value. Activations can discard information even when the calculation is fully deterministic.

4.3 Experiment with the controls

The lab's gate unit receives inputs 2 and 1 with weights 0.5 and -1. Its bias begins at -0.2. Before calculating, notice that the positive and negative contributions exactly cancel. The bias alone decides which side of zero the result occupies in this example.

Changing one quantity at a time makes an experiment interpretable. Record the original inputs and weights. Make a prediction, change just one parameter, then recompute from the start. If you alter both an input and a weight, it becomes harder to attribute the observed change.

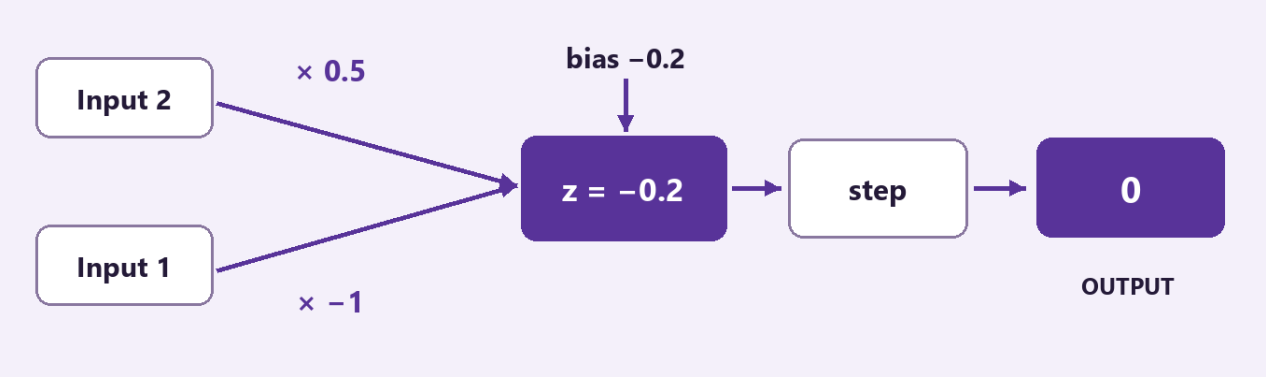


FIG-AI-04-03 The two weighted branches sum to zero. The bias shifts z to -0.2. Under our step convention the final output is zero.

Look closer: In an actual experiment, reset the model before changing a different weight. Otherwise a later trial may include an earlier adjustment, and the apparent effect of one parameter will contain the effects of two.

Find a change that flips the decision

Draw the unit on paper and use movable labels for each parameter. Predict which changes will flip its output. Treat every row below as a fresh trial from the original setup. Record your calculated z before applying the step.

Single change	Products plus bias	Output
None	1 - 1 - 0.2 = -0.2	0
Bias becomes 0	1 - 1 + 0 = 0	1
First weight becomes 0.7	1.4 - 1 - 0.2 = 0.2	1
Second weight becomes -1.3	1 - 1.3 - 0.2 = -0.5	0

Reading the result: Two changes cross the boundary, while the last pushes the score farther below it. A parameter can change the intermediate result without changing the final class. Keeping both values in your record makes that visible.

Try it yourself: How could one unit use the result of another?

Check your reasoning: Treat the first unit's output as a numerical input to the next unit. Volume III, Chapters 5 and 6 traces a small network by repeating the same calculation in a deliberate order.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

For z=-1, 0, 1 compare ReLU with step(z)=1 when z>=0. Explain why a ReLU output of zero does not identify a unique input sum.

Worked guidance

ReLU gives 0, 0, 1; step gives 0, 1, 1. Every nonpositive sum gives ReLU 0. Sigmoid provides a smooth value between 0 and 1, but interpreting it as a useful probability requires an appropriate model and evaluation.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can apply step, ReLU and sigmoid conventions and explain why different sums can share an output. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH05 / CHAPTER 05

Building a Neural Network

BY THE END

Read dependencies, count parameters and explain why nonlinear hidden activations matter.

A network combines simple units. Keep architecture, parameters, activations and output meaning separate.

5.1 Read a network as a dependency map

A recipe cannot use baked bread before the bread has been baked. A feedforward network has the same basic dependency discipline: values in a later layer use completed outputs from an earlier layer. An arrow means this value is used in that calculation. It does not mean the value is consumed or can feed only one destination.

The input layer holds features. A hidden layer computes intermediate values. The output layer computes the final values used for the task. In our two-input, two-hidden, one-output example, both hidden units read both inputs, and the output reads both hidden activations. There is no backward arrow in inference. Training can calculate gradients backward through these dependencies, but that is a different computation from the forward prediction.

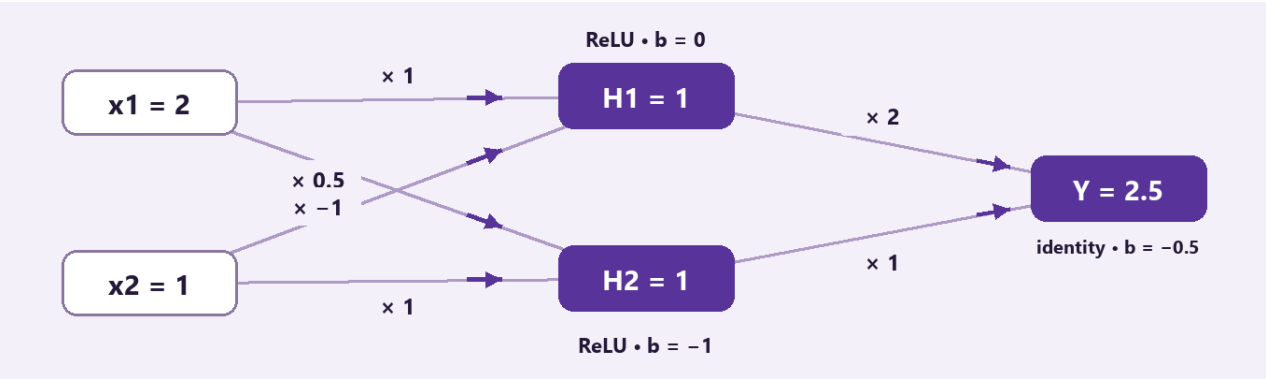


FIG-AI-05-01 The exact worked network used throughout this volume has two inputs, two ReLU hidden units and one identity output. Arrows identify the terms that must be included.

Investigate and explain

List the values needed before calculating h1, h2 and Y. Explain whether h1 must be finished before h2 and whether Y can be finished before either hidden value is known.

Value	Required earlier values	Can be evaluated alongside
h1	x1 and x2	h2
h2	x1 and x2	h1
Y	h1 and h2	Neither hidden calculation can be skipped

Reading the result: The two hidden units are independent once the inputs and parameters are known. They can be calculated in either order. The output waits for both. In a large implementation, parallel hardware may exploit this independence, but the mathematical result follows the same dependencies on paper.

Try it yourself: Does a circle's position above another create a connection?

Check your reasoning: No. Only a specified connection creates a dependency. Read arrows and equations, not vertical position.

5.2 Count parameters without losing the bias

Architecture is the arrangement of units and connections. Parameters are the numbers stored on those connections and in the units. Keeping these separate helps you understand what changes when a model learns: ordinary training changes parameters while the selected architecture stays fixed.

In a fully connected layer, every unit receives every output of the previous layer. If there are m incoming values and k computed units, the layer has m times k weights and k biases, assuming one bias per computed unit. Inputs merely hold features here, so they have no trainable biases. Count each layer separately before adding totals. Parameter count measures stored adjustable numbers, not proof of model quality.

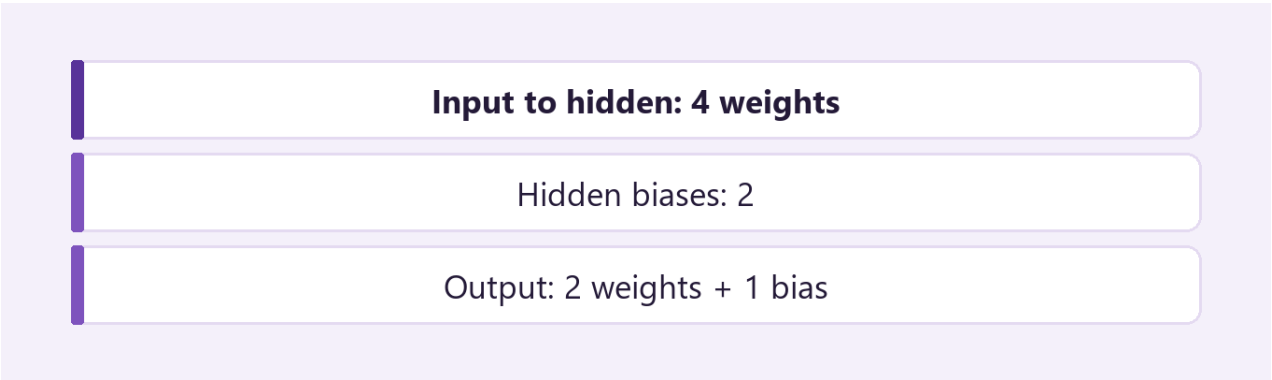


FIG-AI-05-02 The two-input, two-hidden, one-output network has nine parameters. The input values and hidden activations are computed data, not additional trainable parameters.

Investigate and explain

Count the parameters for architectures 2-2-1 and 2-3-1 under the same fully connected convention. Show your decomposition so that a missing bias is visible.

Architecture	First layer	Output layer	Total
2-2-1	2 x 2 + 2 = 6	2 x 1 + 1 = 3	9
2-3-1	2 x 3 + 3 = 9	3 x 1 + 1 = 4	13

Reading the result: The extra hidden unit adds two incoming weights and its bias, plus one outgoing weight: four additional parameters. A larger model can represent more possibilities, but it can also be harder to train or evaluate. Compare a candidate model against a simpler baseline on suitable held-out examples rather than choosing by size alone.

Try it yourself: Does running ten examples through the network create ten new sets of parameters?

Check your reasoning: No. The same stored parameters are reused. Activations change with the examples; the parameters change only when an update procedure changes them.

5.3 Why hidden layers need a nonlinear rule

Two successive clerks can sometimes be replaced by one equivalent clerk. If the first always doubles a number and adds one, and the second triples that result and adds four, the entire chain is simply six times the original number plus seven. Adding a layer did not create a fundamentally new kind of rule.

This collapse also occurs when every layer uses only weighted sums, biases and identity activations: composing affine rules produces another affine rule. A nonlinear activation such as ReLU changes that. ReLU keeps positive inputs and replaces negative inputs with zero, so different regions of input can behave differently. Nonlinearity makes layered models more expressive, although it does not guarantee that training will find a good model.

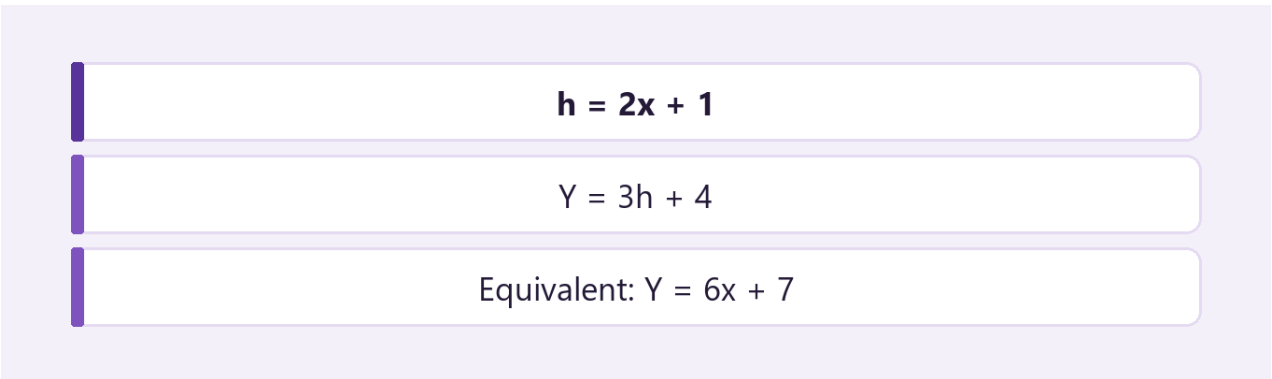


FIG-AI-05-03 Substitute the first expression into the second. The boxes show an identity-activation chain; replacing h with ReLU(2x + 1) breaks the single-affine simplification across all inputs.

Investigate and explain

Calculate the identity chain for x = -1, 0 and 1. Repeat with h = ReLU(2x + 1). Identify the input where the hidden activation changes the result.

x	Identity h	Identity Y	ReLU h	ReLU Y
-1	-1	1	0	4
0	1	7	1	7
1	3	13	3	13

Reading the result: Only x = -1 makes the hidden weighted sum negative in these three trials, so only that row changes. The identity chain gives 1 there; the ReLU chain gives 4. This concrete counterexample explains why activation is more than a decorative extra box. To understand a network, trace both the weighted sum and its transformed output.

Try it yourself: Can several identity layers still be useful for other implementation reasons?

Check your reasoning: They may be used in particular designs, but as a pure composition of affine maps they do not enlarge the set of input-output affine functions. This chapter is explaining representational power, not every engineering use.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Draw a fully connected 2-3-1 architecture, count its trainable parameters with one bias per computed unit, and explain why adding identity-only layers does not produce a general nonlinear rule.

Worked guidance

The hidden layer has 6 weights+3 biases; the output has 3 weights+1 bias, for 13. Composed weighted sums and biases remain affine. A nonlinear activation such as ReLU allows behavior that cannot be collapsed to one affine rule over all inputs.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can read dependencies, count parameters and explain why nonlinear hidden activations matter. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH06 / CHAPTER 06

Forward Propagation

BY THE END

Compute a complete two-layer forward pass and distinguish raw scores from decisions.

Use activated hidden values as the inputs to the next layer. Each arrow contributes one paired product.

6.1 Calculate the hidden layer

Let the inputs x1 and x2 be 2 and 1. H1 multiplies them by 1 and -1, adds zero bias, and applies ReLU. H2 uses weights 0.5 and 1, bias -1, and also applies ReLU. Each unit receives the same pair of inputs through its own pair of weights.

A hidden unit's pre-activation sum and its output can differ. Label them separately even when ReLU happens to leave the sum unchanged. Later a negative sum will expose why passing the activated output matters.

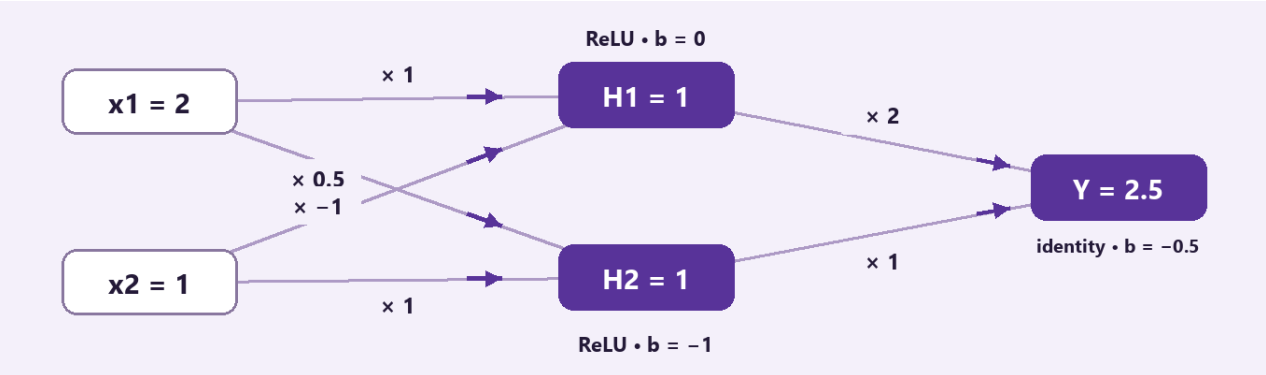


FIG-AI-06-01 The upper hidden unit gives ReLU(2 - 1) = 1. The lower gives ReLU(1 + 1 - 1) = 1. Equal outputs here arise from different calculations.

Look closer: A hidden unit may stay at zero over several input changes because its sum stays negative. The incoming information still changed. Keeping the raw sum in the trace reveals what the zero output conceals.

Two assistants see the same inputs differently

Make a separate worksheet row for each unit. Now rerun both rows for inputs 1 and 2, with every parameter held fixed. Predict whether the upper unit is still positive.

Inputs	Unit	Sum before ReLU	Output
2, 1	H1	2 - 1 + 0 = 1	1
2, 1	H2	1 + 1 - 1 = 1	1
1, 2	H1	1 - 2 + 0 = -1	0
1, 2	H2	0.5 + 2 - 1 = 1.5	1.5

Reading the result: H1 acts as a one-sided difference detector in this example. A negative difference gives output zero, so the final layer will receive zero along that branch. Calling this a "detector" describes its arithmetic; no training experiment has established what it means in a real application.

Try it yourself: Why would calculating H2 from H1's value give the wrong answer?

Check your reasoning: The units lie in the same layer with no arrow between them. Both read the original inputs. H2 therefore needs x1 and x2 regardless of which hidden unit you calculate first.

Another case: Make both assistants quiet

Use x1 = 0 and x2 = 0. Compute H1 and H2 with the weights on this page. Predict the identity output with final bias -0.5 before calculating any final multiplication.

Feedback: H1 receives sum 0 and outputs 0. H2 receives sum -1 and outputs 0. Both outgoing contributions therefore vanish, leaving Y = -0.5. A zero hidden vector does not force a zero final output when the final unit has its own bias.

6.2 Finish one complete forward pass

The output unit uses hidden outputs h1 and h2 with weights 2 and 1, then adds bias -0.5. Its activation is the identity rule, which simply returns the sum. For the original inputs 2 and 1, the hidden layer supplied 1 and 1. Predict whether the final output exceeds two.

Forward propagation evaluates the required quantities in dependency order. The final score is 2 × 1 + 1 × 1 - 0.5 = 2.5. During this calculation the weights are read and held fixed. Producing a new prediction therefore requires no training update.

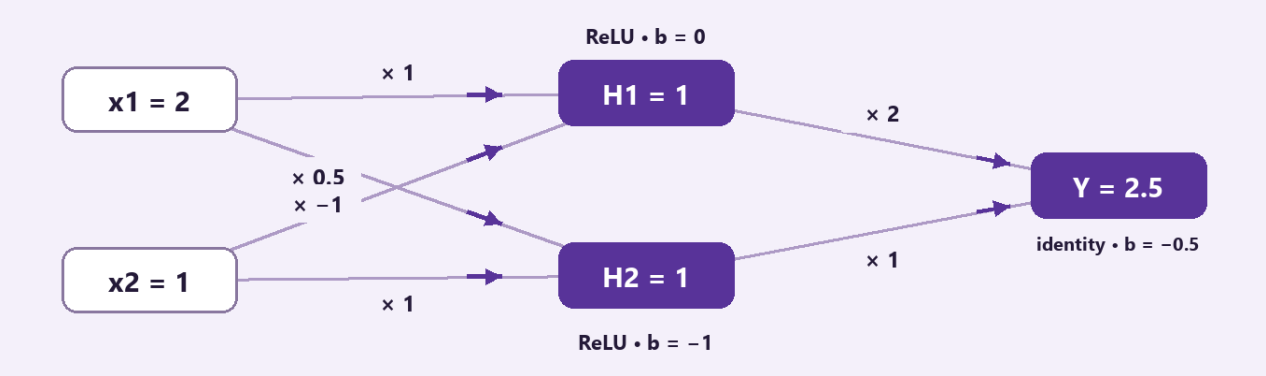


FIG-AI-06-02 The upper and lower hidden outputs both equal 1. Their outgoing weights turn them into contributions of 2 and 1. The output's -0.5 bias leaves 2.5.

Look closer: A zero contribution from one hidden unit leaves the other branch and output bias active. A single silent unit does not imply that the entire network has stopped producing a numerical output.

Trace every number to the final score

Cover the final score in the diagram. Reconstruct it from the four incoming weights, both hidden biases, the two outgoing weights and the final bias. Check the intermediate values in the order below.

Stage	Computation	Result
Hidden H1	ReLU(1 × 2 - 1 × 1 + 0)	1
Hidden H2	ReLU(0.5 × 2 + 1 × 1 - 1)	1
Output Y	Identity(2 × 1 + 1 × 1 - 0.5)	2.5

Reading the result: Each hidden activation is applied before its result is sent forward. Adding all nine parameters and inputs into one long sum ignores their pairings and dependency structure. If a result disagrees, compare the two hidden values before revisiting the final layer.

Try it yourself: For the second input pair 1 and 2, what is Y?

Check your reasoning: The hidden outputs are 0 and 1.5. The identity output is 2 × 0 + 1 × 1.5 - 0.5 = 1.0. The architecture and parameters are identical across the two cases.

Another case: One changed outgoing weight

Return to inputs 2 and 1, where both hidden outputs equal 1. Change only the outgoing weight from H1 from 2 to 3. Write down the first quantities that can change and recompute Y.

Feedback: Both hidden outputs remain one because their incoming parameters are untouched. The contribution from H1 changes from two to three. Y becomes 3 + 1 - 0.5 = 3.5. Following the arrow direction limits how much of the calculation must be redone.

6.3 What should a score mean?

A score of 2.5 has no units until the designer defines the prediction task. Regression estimates a numerical quantity, such as waiting time in minutes. Classification chooses among categories, such as a routine or urgent message. A score can support either task with an appropriate design.

For a toy classifier, declare "flag for review when Y ≥ 2." This decision rule turns a numerical output into a class. Setting the threshold changes how many messages are flagged. A regression model predicting waiting time would need meaningful time labels and validation against observed waits.

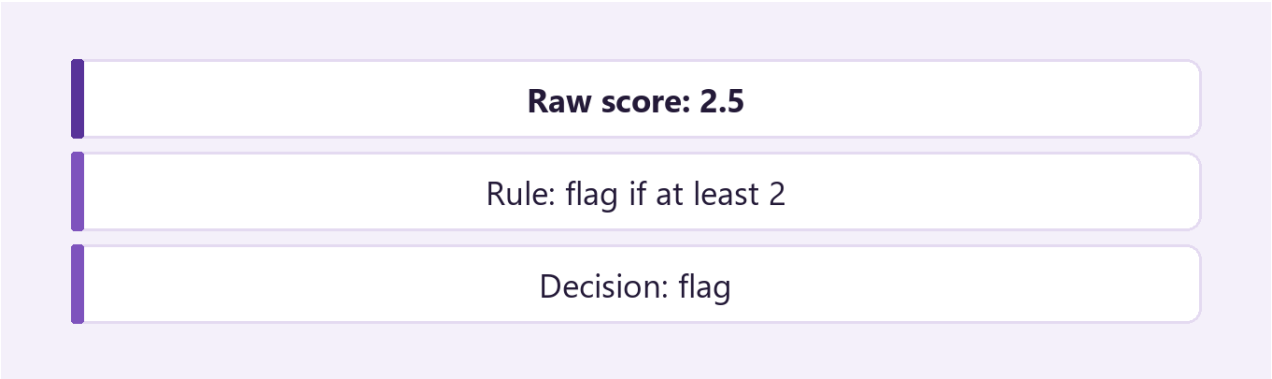


FIG-AI-06-03 The network produces the first value. A separate documented rule interprets it. This example supplies no evidence that 2.5 is a duration or a calibrated probability.

Look closer: A classifier may use more than two categories. For this two-way flag we chose one score and a threshold so the interpretation can be reproduced on paper. Always specify which class corresponds to a positive decision.

Give the output an explicit job

Compute the decisions for scores 1.0, 2.0 and 2.5 under thresholds 2 and 2.4. Imagine the clerk can review only a few cases and describe what happens near the cutoff.

Score	Flag at ≥ 2?	Flag at ≥ 2.4?
1.0	No	No
2.0	Yes	No
2.5	Yes	Yes

Reading the result: Raising the cutoff removes a flag at 2.0 without changing a single weight or score. Whether this is acceptable depends on which true cases are missed and the costs of unnecessary reviews. Score arithmetic alone cannot answer that question.

Try it yourself: If the task is predicting a wait in minutes, what would a label look like?

Check your reasoning: It could be the actual elapsed minutes from request to pickup for an observed case. A flag such as "urgent" would answer a different prediction question.

Another case: A threshold that includes equality

Three requests receive scores 1.9, 2.0 and 2.1. Compare a flag rule using > 2 with the book's ≥ 2 rule. Identify exactly which visitor experiences a different decision.

Feedback: Only the visitor whose score equals 2.0 changes. The strict rule flags only 2.1, while the inclusive rule also flags 2.0. Recording the comparison convention makes the decision reproducible even when scores are displayed rounded to a few decimals.

6.4 A change at one input

A network's drawing can make everything look interconnected. To understand an individual prediction, change one input and trace the numerical consequences one layer at a time. Keeping the weights fixed isolates what the input change does in this particular network.

Fix x2 at 1 and vary x1. H1 computes ReLU(x1 - 1). H2 computes ReLU(0.5 × x1 + 1 - 1). The first hidden unit begins passing a positive contribution only when x1 exceeds one. The final score can therefore change at different rates in different regions.

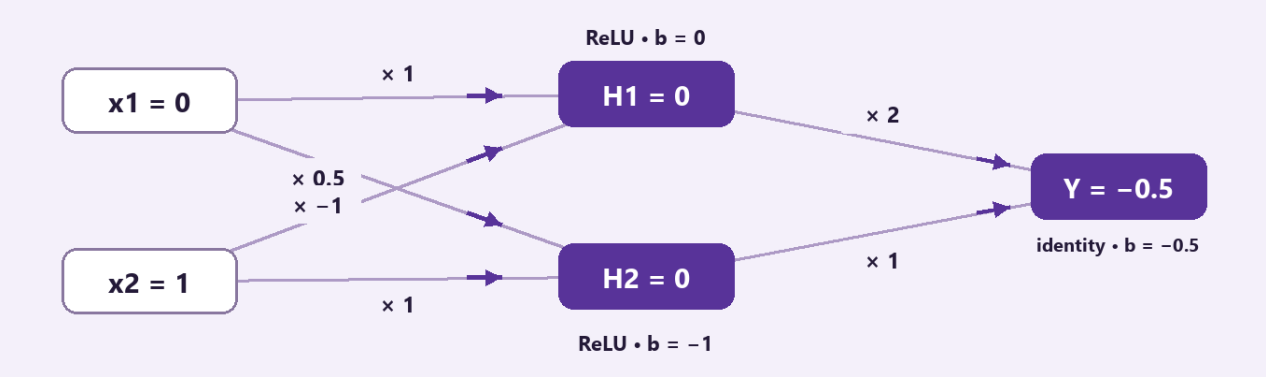


FIG-AI-06-04 For x1 = 0 and x2 = 1, both hidden outputs are zero and the final bias alone gives -0.5. The identity output permits a negative score.

Look closer: Repeating the same deterministic forward pass is a useful consistency check. If the same inputs and parameters produce a different hand calculation, compare the first hidden sum on which your two traces disagree.

Locate where two predictions diverge

Build the entire network on paper. Choose x1 equal to 0, 1, 2 and 3 in turn. Before each trial predict which hidden units will be positive. Record their values, Y and the decision at threshold 2.

x1	h1	h2	Y
0	0	0	-0.5

x1	h1	h2	Y
1	0	0.5	0
2	1	1	2.5
3	2	1.5	5

Reading the result: Only the final two trials are flagged. The first change of one input unit increases Y by 0.5. The next increases it by 2.5 because H1 starts contributing too. This bend is one reason nonlinear activations give networks expressive power.

Try it yourself: If the four predictions are wrong on labelled cases, which step would actually change the network?

Check your reasoning: A training algorithm can use prediction errors to adjust parameters. Merely repeating the forward pass repeats the same outputs under the same settings.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

For x1=1, x2=2 compute h1=ReLU(x1-x2), h2=ReLU(0.5x1+x2-1), Y=2h1+h2-0.5. Repeat with x1=3. Use threshold Y>=2 and explain whether training occurred.

Worked guidance

First h1=0, h2=1.5, Y=1; second h1=1, h2=2.5, Y=4. Only the second passes the threshold. Parameters did not change, so this is an input experiment during inference, not training.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can compute a complete two-layer forward pass and distinguish raw scores from decisions. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

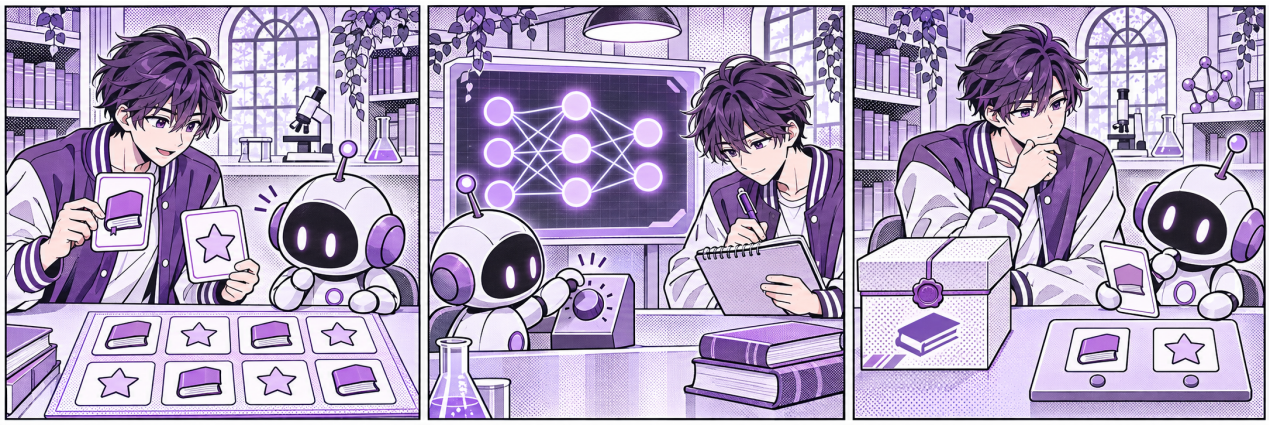
AI-CH07 / CHAPTER 07

How Neural Networks Learn

BY THE END

Calculate simple loss and one gradient-based update; distinguish backpropagation from optimization.

Training supplies a reason and a method for changing parameters. A lower training loss is useful evidence, but incomplete evidence.



ART-AI-CH07 Examples, parameter adjustment and unfamiliar cases play different roles. The control dial is an analogy; the worked training update states the numerical rule.

7.1 Put a number on a prediction error

Pip predicts a four-minute wait. The visitor actually waits six. Before changing anything, decide how to quantify this discrepancy. A loss is a number describing how far a prediction misses the desired outcome under a chosen rule. Smaller loss is better under that rule.

Squared error subtracts the observed value from the prediction and squares the difference, meaning multiply it by itself. Both a two-minute overestimate and a two-minute underestimate produce loss four. Large errors receive extra emphasis. Mean squared error averages this quantity across examples.

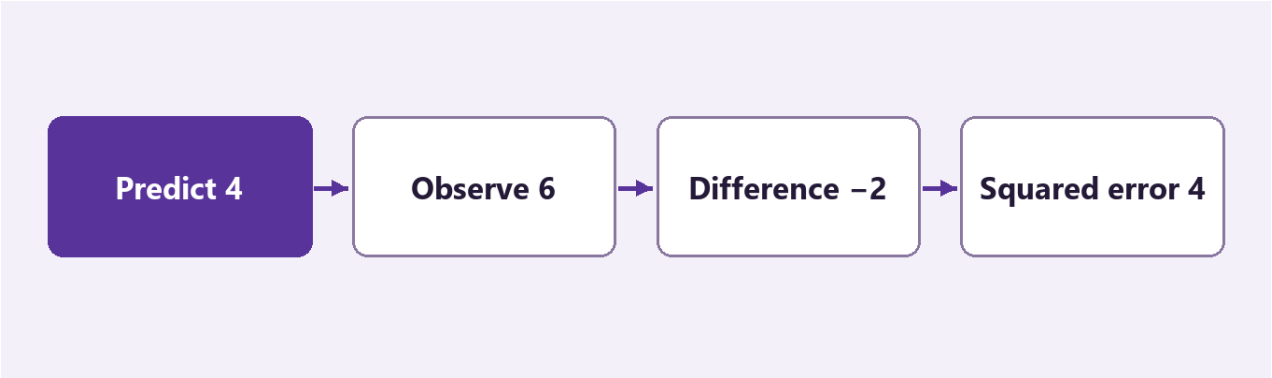


FIG-AI-07-01 Keep the actual wait separate from the prediction. A label is an observed target used for comparison and can itself contain measurement error.

Look closer: For squared error, a discrepancy of four contributes sixteen units of loss, while two discrepancies of two contribute eight together. Choosing a loss expresses how different mistakes should be weighted.

Compare two models on the same labels

For two actual waits 2 and 6, model A predicts 3 and 5. Model B predicts 2 and 4. Predict which has lower mean squared error, then calculate. Keep signed errors visible before squaring.

Model	Errors	Squared errors	Mean
A	+1, -1	1, 1	1
B	0, -2	0, 4	2

Reading the result: A has lower mean squared error despite never hitting a label exactly. B is exactly right once and misses by two once. Changing the loss can change the preference between models: under mean absolute error both average one minute of error here.

Try it yourself: Why is the signed average error insufficient for model A?

Check your reasoning: Its +1 and -1 cancel to zero even though both predictions are wrong. The signed average describes a tendency to over- or underestimate, while squared error measures discrepancy.

Another case: Errors with different units

A wait-time model has errors of 1 and -3 minutes. Calculate its mean squared error. If another report describes average absolute error, what number would it give and in what units?

Feedback: Squared errors are 1 and 9, so the mean is 5 squared minutes. Absolute errors are 1 and 3, averaging 2 minutes. The numbers summarize different consequences and have different units. Always name the loss before comparing models or interpreting an improvement.

7.2 Learning is an update loop

A clerk adjusting a shelf scale asks which direction reduces its error. Training similarly uses examples, a forward prediction, loss, and a parameter update. An epoch is one pass through the training examples. Many epochs may be required, and additional epochs can eventually hurt performance on unfamiliar cases.

A gradient describes how loss changes locally as each parameter changes. Backpropagation efficiently computes these sensitivities by working backward through the network's dependencies. An optimizer uses the sensitivities to choose updates. A learning rate sets the scale of a step. This explanation needs no calculus to distinguish measuring a direction from taking a step.

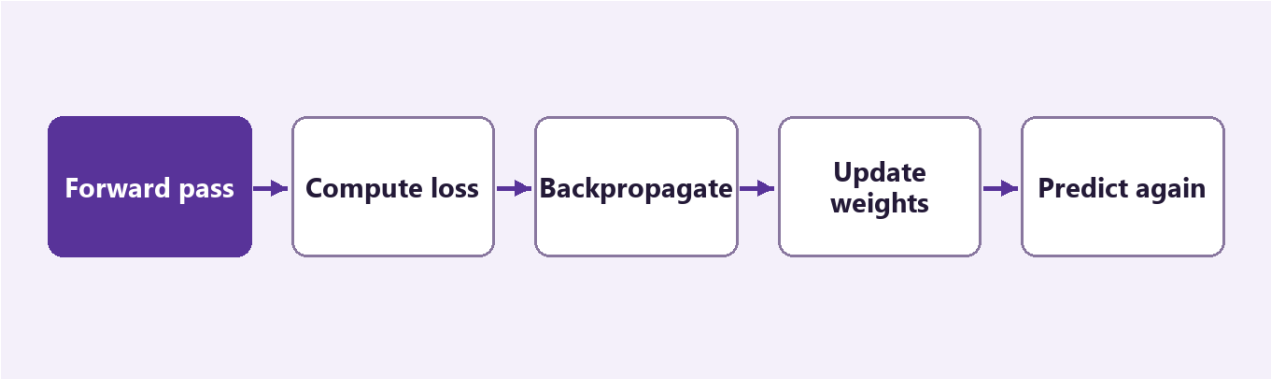


FIG-AI-07-02 Only the update box changes the stored parameters. The backward computation describes how the output error depends on earlier weights through their connections.

Look closer: A learner might keep trying parameter settings until a single labelled case fits exactly. That establishes a fit on that case. It says little about how an update affects other cases sharing the same parameters.

Distinguish backpropagation from the update

Try a one-weight model with prediction equal to w × 2, target 4 and starting w = 1. On paper compare several proposed weights by fresh forward calculations. This trial table illustrates a loss surface without pretending to implement backpropagation.

w	Prediction	Squared loss
1.0	2	4
1.5	3	1
2.0	4	0
3.5	7	9

Reading the result: An increase toward two reduces loss here; a sufficiently large increase overshoots and makes the result worse. Fitting this one example perfectly gives no guarantee about a second visitor. In a real model parameters are shared across many predictions, so an update can benefit one example and harm another.

Try it yourself: Does the term epoch mean one weight update?

Check your reasoning: Only in special setups. One pass through a training set may contain several batches and therefore several updates. State the batch policy before comparing update counts.

Another case: Rehearsal on a second example

The one-weight model w × x fits x = 2, target 4 exactly at w = 2. A second labelled case has x = 1, target 3. Calculate its loss at the same weight and the mean squared loss over the two cases.

Feedback: The second prediction is 2, giving squared error 1. The first squared error is zero, so the two-case mean is 0.5. Adjusting w solely to perfect the second case can spoil the first. A shared parameter requires a training objective across examples.

7.3 One parameter update you can audit

Imagine adjusting a dial that controls a prediction. You compare the prediction with a target, calculate a loss and ask which way a small turn would reduce that loss. A gradient is a local sensitivity: how the loss changes as a parameter changes by a small amount. You can use a supplied gradient without first learning calculus.

For this miniature model, prediction = w times x, with x = 2 and target 4. The loss is the squared error, (prediction - target) squared. At w = 1 the prediction is 2 and the loss is 4. The loss gradient with respect to w is -8; for this particular formula it can be calculated as 2 times (prediction - target) times x. Gradient descent updates w to w - learning_rate times gradient. The learning rate controls the step size.

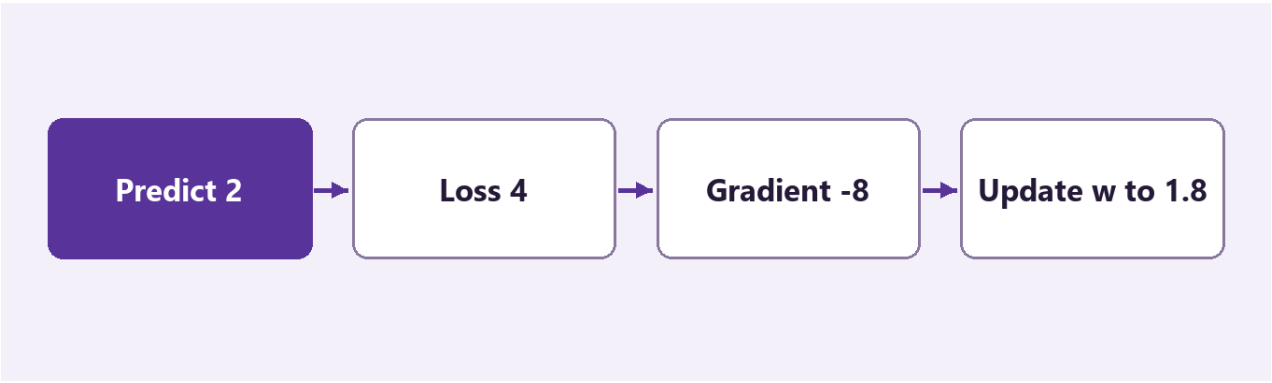


FIG-AI-07-03 With learning rate 0.1, the update is 1 - 0.1 x (-8) = 1.8. The next forward pass must use the new weight.

Investigate and explain

Verify the update using only arithmetic. Calculate the next prediction and loss. Then repeat from the original w = 1 using learning rate 1 instead of 0.1. Keep the starting state fixed when comparing rates.

Rate	New w	New prediction	New loss
0.1	1.8	3.6	(3.6 - 4)^2 = 0.16
1.0	9	18	(18 - 4)^2 = 196

Reading the result: The smaller step reduces loss in this example. The larger step overshoots and makes it much worse. This shows why subtracting a gradient is not a universal guarantee that any step size improves a model. The gradient is local information. The example contains one training case and says nothing about generalization to new inputs.

Try it yourself: Why subtract a negative gradient?

Check your reasoning: A negative gradient says that a small increase in w decreases loss locally. Subtracting that negative value increases w, moving in the downhill direction for a sufficiently small step.

7.4 Backpropagation, batches and epochs

A network can contain many parameters, so turning one dial at a time and recomputing every effect is inefficient. Backpropagation works backward through the dependencies to calculate loss gradients. The optimizer then uses those gradients to update parameters. These two jobs are related but distinct.

A batch is a group of training examples processed for an update. An epoch is one pass through the chosen training dataset. With 12 examples and batches of 4, one epoch uses three batches if each example is used once. Training can repeat epochs. Validation can compare saved checkpoints, but final test data should not be repeatedly used to choose the checkpoint. Different starting parameters or example orders can produce different runs; record them when comparing experiments.

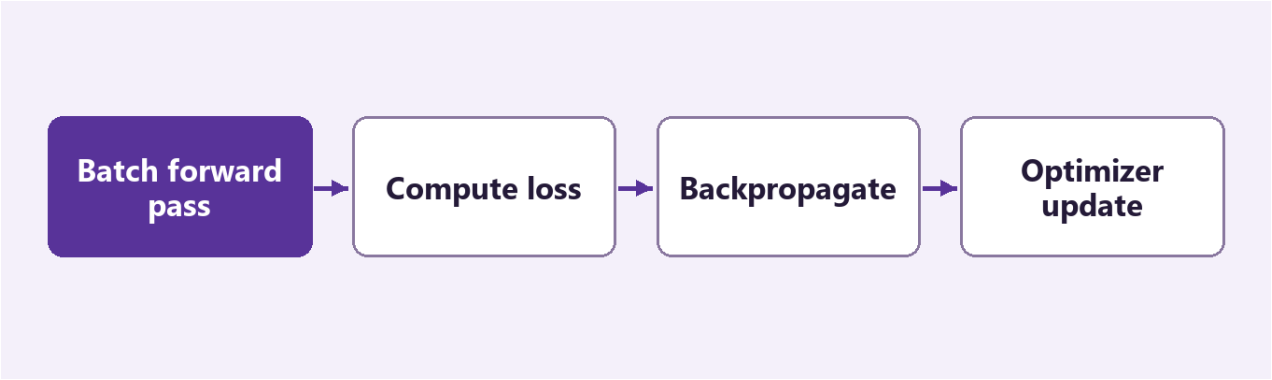


FIG-AI-07-04 The loop repeats over batches. The prediction flows forward, gradient calculation flows backward through dependencies, and the optimizer changes stored parameters.

Investigate and explain

Plan two epochs over 12 examples using batches of 4. Count batches and updates. Suppose validation loss at the ends of epochs 1, 2 and 3 is 0.42, 0.31 and 0.38. Choose a checkpoint under the rule lowest validation loss.

Quantity	Result	Reason
Batches per epoch	3	12 divided by 4
Updates in two epochs	6	One per batch
Chosen checkpoint	Epoch 2	0.31 is the lowest supplied validation loss

Reading the result: The number of updates is six under the stated convention, not 24 and not two. The best supplied validation value occurs at epoch two even though epoch three trained longer. These numbers are fictional teaching data. In an actual experiment, preserve the observed record and avoid treating a single run as proof that one setup is always best.

Try it yourself: Does backpropagation itself choose the learning rate?

Check your reasoning: No. It computes gradients. The optimizer and its configured settings determine how those gradients change parameters.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

For prediction w*x with x=2, target 4, w=1, use gradient-8 and learning rate 0.1. Compute the updated weight and loss. Explain what a batch and epoch mean for 12 cases in batches of 4.

Worked guidance

New weight 1.8, prediction 3.6, squared loss 0.16. A batch has 4 cases; one epoch uses 3 batches and, with one update per batch, 3 updates. The one-case improvement does not establish performance on unseen cases.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can calculate simple loss and one gradient-based update; distinguish backpropagation from optimization. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH08 / CHAPTER 08

Evaluating a Model and Generalization

BY THE END

Use held-out evidence, learning curves and confusion-matrix metrics without overstating conclusions.

A model is valuable only if its behavior is useful beyond the examples used to develop it.

8.1 Keep some examples unseen

A model can memorize peculiarities of its training examples. Generalization means performing well on relevant new examples. Reserve training data for parameter updates, validation data for choices such as stopping time, and test data for a final evaluation after choices are fixed.

Overfitting occurs when a model increasingly fits the training sample at the expense of performance on new data. In a learning curve, falling training loss alongside rising validation loss is a clue. Small validation sets can be noisy, so inspect the evidence and the way examples were collected before treating one change as decisive.

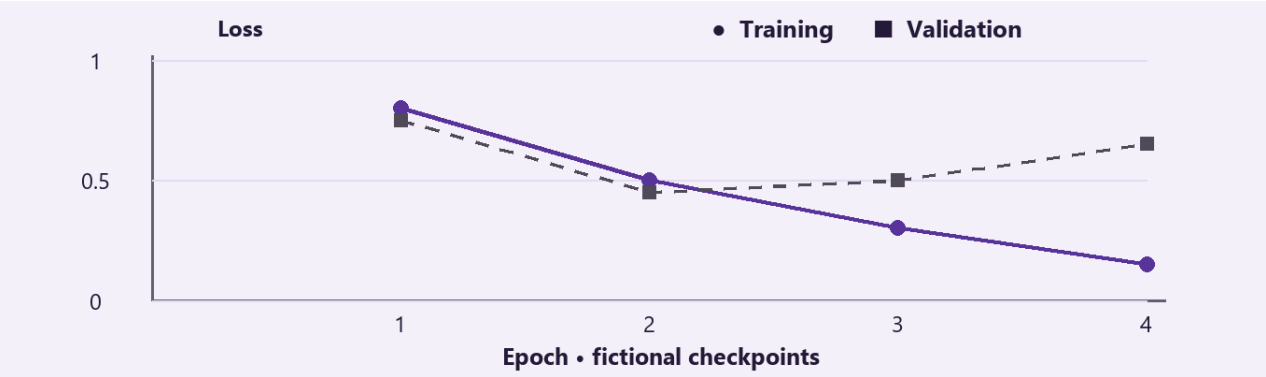


FIG-AI-08-01 Fictional losses at four epochs illustrate a widening gap. Training decreases at every checkpoint. Validation is lowest at epoch two. The series use different line styles as well as labels.

Look closer: Suppose validation performance keeps worsening while training performance improves. Before changing the architecture, inspect split quality and labels. A curve suggests a question; it cannot identify every cause by itself.

Read the turn in a learning curve

A team plans to select one of four saved checkpoints using validation loss. Predict which it should select from the table. Then explain why evaluating the chosen checkpoint on a separate test set supplies different evidence.

Epoch	Training loss	Validation loss
1	0.80	0.75
2	0.50	0.45
3	0.30	0.50
4	0.15	0.65

Reading the result: Epoch two has the lowest listed validation loss. A separate test measures the model after this selection. Choosing the epoch with the best test score would turn the test set into another selection set. These values were constructed for reading practice and are not experimental measurements.

Try it yourself: Could copies of the same photo in training and test create a reassuring result?

Check your reasoning: Yes. Near-duplicate pictures make supposedly unseen examples unusually familiar. Split related examples together and check that the held-out cases resemble the intended use.

Another case: When a test set becomes familiar

Three teams repeatedly try new architectures and report only the best score on one shared test set. What role is that set playing in practice, and what should a final evaluation do differently?

Feedback: The test outcomes are guiding model selection, so the set is effectively validation data. Decide the model and procedure using training and validation evidence. Then evaluate once on genuinely held-out, appropriately sampled cases and report how those cases were selected.

8.2 Look past overall accuracy

An image classifier marks a scanned page as clear or blurred. Suppose only one in ten pages is blurred. A system that always answers “clear” achieves 90% accuracy on such a batch, yet it never finds the pages needing another scan. Accuracy is the fraction of predictions that match their labels.

For a chosen positive class, a false negative misses a real positive and a false positive flags a negative case. A confusion matrix counts actual classes against predicted classes. Read its axes carefully. It lets a team inspect errors that a single accuracy number hides.

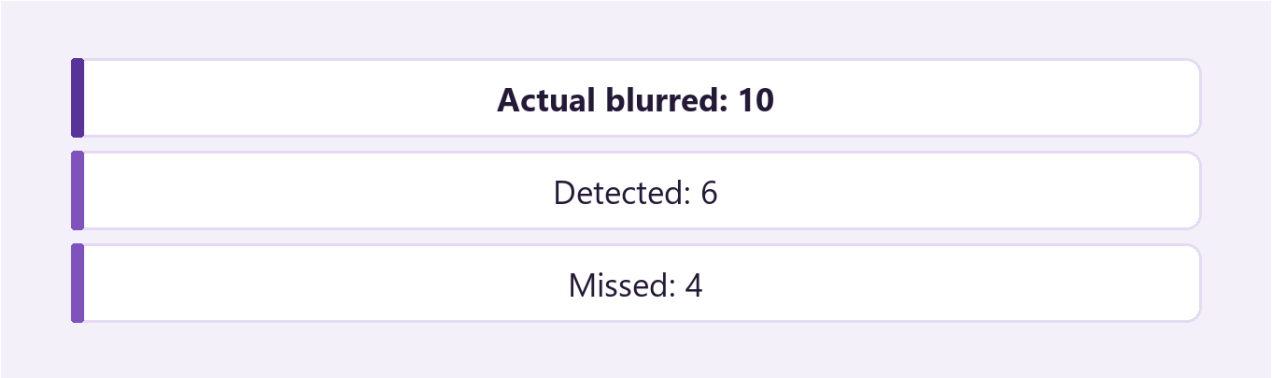


FIG-AI-08-02 In a fictional set of 100 scans, six of ten blurred pages are detected. The two classes are unequal in size, so the majority of cases contribute clear-page decisions.

Look closer: A false alarm and a missed urgent notice can have very different consequences. Include their counts alongside an aggregate metric so someone making the service decision can see the trade-off.

Find the errors hidden in one percentage

Suppose the classifier also flags eight of the ninety clear pages. Fill the four cells for actual and predicted classes. Compute accuracy and the portion of truly blurred pages found, called recall for the blurred class.

Actual class	Predicted blurred	Predicted clear
Blurred	6	4
Clear	8	82

Reading the result: Accuracy is (6 + 82) / 100 = 88%. Blurred-page recall is 6 / 10 = 60%. The always-clear baseline has 90% accuracy but zero blurred-page recall. Choosing a policy requires considering missed rescans, unnecessary rescans and the consequences for users.

Try it yourself: How might a text classifier face a similar problem?

Check your reasoning: An urgent-message detector can seem accurate on a mailbox full of routine notes while missing every urgent one. Evaluate urgent misses and false alarms separately, using fresh labelled messages.

Another case: Another overall score that conceals a cost

Of twenty truly urgent notes a classifier flags fifteen. It also flags ten routine notes. Compute urgent recall and the fraction of its flags that are actually urgent. Which quantity would a busy reviewer especially notice?

Feedback: Recall is 15/20 = 75%. Of 25 total flags, 15 are truly urgent, so 60% of flags are correct, a quantity called precision. A reviewer sees the burden of the ten unnecessary checks; a missed visitor experiences one of the five urgent failures. Both perspectives matter.

8.3 Read the errors behind accuracy

The library flags urgent messages. Correctly ignoring ninety routine messages can make a system look good even if it misses all ten urgent ones. Accuracy answers how many predictions are correct overall. It does not tell us which mistakes occur or whom they affect.

Choose urgent as the positive class. A true positive is an urgent message flagged urgent; a false negative is an urgent message missed. A false positive is a routine message flagged urgent, and a true negative is a routine message correctly left routine. Recall divides true positives by all actual positives. Precision divides true positives by all predicted positives. A confusion matrix arranges these counts by actual and predicted class. Always name the row and column meanings.

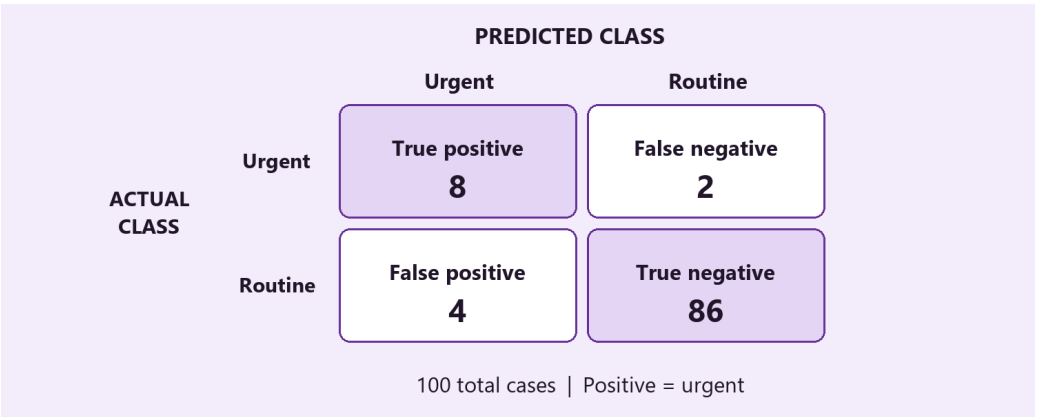


FIG-AI-08-03 Rows are actual classes and columns are predictions. With TP=8, FN=2, FP=4 and TN=86, every one of the 100 examples appears in exactly one cell.

Investigate and explain

Calculate accuracy, urgent recall and urgent precision for the displayed matrix. Then compare the always-routine baseline. Decide which count a library worried about missed urgent requests should inspect first.

Measure	Model calculation	Always routine
Accuracy	(8 + 86)/100 = 94%	90/100 = 90%
Urgent recall	8/(8 + 2) = 80%	0/10 = 0%
Urgent precision	8/(8 + 4) = 66.7%	Undefined: no positive predictions

Reading the result: The model detects eight urgent cases but misses two and creates four false alarms. The baseline has no positive predictions, so its precision denominator is zero; do not report a fabricated measured precision. Software may apply a documented convention for that case. A deployment choice depends on the consequences of both kinds of error and the availability of human review.

Try it yourself: Does high recall guarantee high precision?

Check your reasoning: No. Flagging every message gives perfect recall when positives exist, but may create many false positives and low precision.

8.4 Make a split match the future question

Imagine testing whether a tutor can teach new students while putting different essays from the same students in both practice and test piles. The test may partly measure familiarity with those writers. The way examples are split should match the generalization claim you want to make.

If the target is performance on new readers, keep each reader's related records together when splitting. If the target is future requests, a time-based split may better represent change over time. Leakage means evaluation has access to information that would not legitimately be available at prediction time, directly or through preparation. Fitting preprocessing on all examples before splitting can leak information as well. No single split guarantees future performance; check representativeness and document the population.

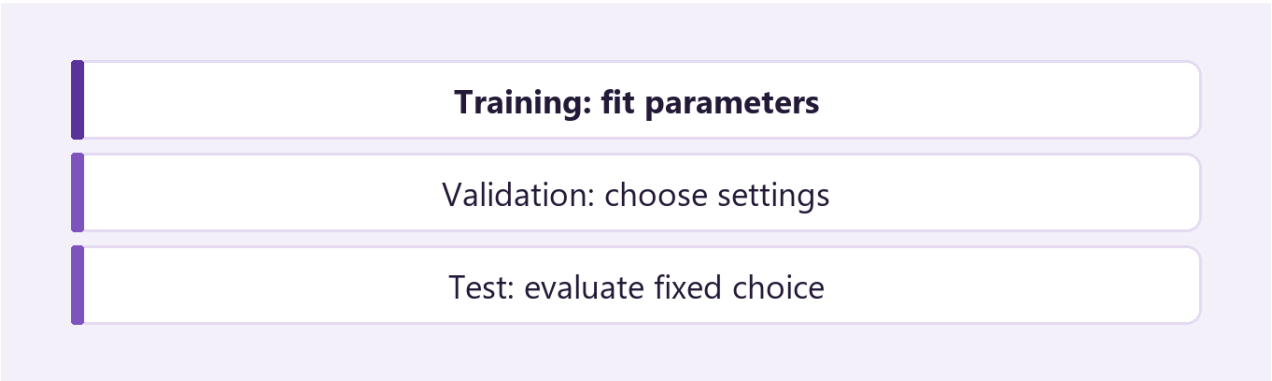


FIG-AI-08-04 These are roles, not magical percentages. Data grouping and timing matter alongside how many examples go into each set.

Investigate and explain

Readers A and B each contribute two ratings. The goal is to estimate performance on a new reader. Compare a random rating split with a grouped reader split. State what would still be weak about a test containing only one reader.

Split	Training	Test	Main interpretation
By rating	A1, B1	A2, B2	More ratings from familiar readers
By reader	A1, A2	B1, B2	One unseen reader, very limited evidence

Reading the result: The grouped split better matches the stated new-reader question, but one test reader is far too narrow to establish broad reliability. More representative independent evidence is needed. A clean split avoids one source of misleading evaluation; it does not make the sample automatically large or diverse. Always say what the observed score can and cannot support.

Try it yourself: If the team repeatedly chooses features using test results, is that still a final independent test?

Check your reasoning: No. Those results have influenced the design. Use validation for such choices and obtain suitable fresh held-out evidence for a final independent assessment.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Using TP 8, FN 2, FP 4, TN 86, compute accuracy, recall and precision. Describe a split for measuring performance on new readers and explain why selecting by final test scores weakens independence.

Worked guidance

Accuracy 94%, recall 80%, precision 8/12=66.7%. Keep related records from each reader together for the new-reader question. Use validation for selection; preserve suitable test data for a fixed final choice. Report sample limits and relevant error consequences.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can use held-out evidence, learning curves and confusion-matrix metrics without overstating conclusions. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

AI-CH09 / CHAPTER 09

Neural Networks in Everyday Applications

BY THE END

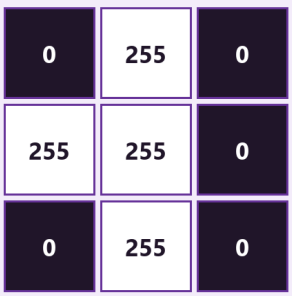
Translate images and text into defined inputs and place prediction within a checkable service.

The same input-to-output discipline supports different applications, but their data, targets and failure consequences differ.

9.1 Images become arrays before predictions

A scanned page reaches a computer as numbers representing pixels, small picture elements. A grayscale pixel might record brightness from 0 to 255 under a chosen convention. A simple model can flatten a small grid into a sequence, while image-oriented architectures can preserve local spatial relationships.

A convolutional neural network is an optional preview: it applies learned small filters across image locations, sharing parameters across positions. This can make local visual patterns easier to represent than treating every location independently. It is still trained from examples and can fail when lighting, orientation, handwriting or scan quality differs from training data. A predicted digit is a class decision, not a guaranteed transcription.



ROW-MAJOR ORDER

[0, 255, 0,
255, 255, 0,
0, 255, 0]

9 pixels -> 9 input values

FIG-AI-09-01 This small 3-by-3 grid is a toy brightness pattern, not a real digit-recognition dataset. Row-major order reads each row left to right before moving down.

Investigate and explain

Flatten the grid [[0,255, 0],[255,255, 0],[0,255, 0]] in row-major order. Identify the number of inputs. Then rotate or move the pattern on paper and explain what changes for a model that expects fixed positions.

Representation	Values	Interpretation
Row 1	0, 255, 0	First three inputs
Row 2	255, 255, 0	Next three inputs
Row 3	0, 255, 0	Last three inputs
Flattened length	9	One input per pixel

Reading the result: The flattened sequence has nine entries. Moving the visible pattern changes which positions are bright, even if a person recognizes the same intended shape. Preprocessing, architecture and representative training examples can help, but performance still needs evaluation. In a library workflow, an uncertain scanned copy ID should be checked against the catalogue and corrected by a person before a wrong copy is assigned.

Try it yourself: Does a high digit score prove that the physical copy exists?

Check your reasoning: No. Recognition estimates what the image says. Exact lookup and availability checks answer separate questions about the library.

9.2 Text and recommendations need a defined target

A model cannot act on the meaning of a book title until the system represents that text numerically. A simple classroom representation records whether selected words appear. More advanced systems use embeddings, learned numerical vectors that represent items or tokens through patterns in data. A token is a unit used by a text-processing system; it may be a word or part of a word.

For this chapter, use only three indicators: contains map, contains river and contains poetry. This loses word order and most meaning, but makes the representation inspectable. Recommendation also needs a target: predicting a later voluntary like differs from predicting whether a book will be clicked. Training on clicks can reward attention without demonstrating satisfaction. Choose the measurement to match the intended benefit.

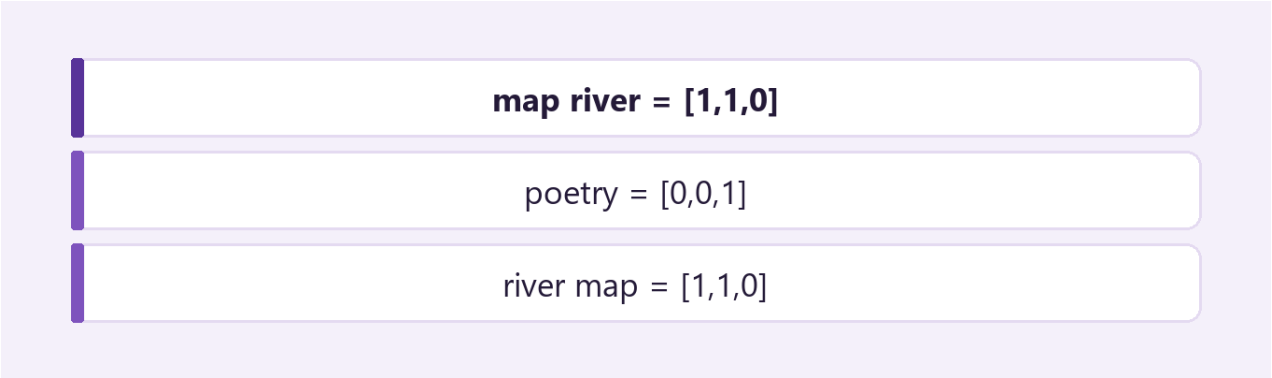


FIG-AI-09-02 The fixed indicator order is map, river, poetry. The first and third texts map to the same vector, showing information that this representation discards.

Investigate and explain

Encode river poetry and then poetry river. Explain one question this representation can answer and one it cannot. Design a baseline recommendation that does not need training.

Text	Indicators [map, river, poetry]	Information lost
river poetry	[0, 1, 1]	Word order
poetry river	[0, 1, 1]	Word order
map	[1, 0, 0]	Context and intent

Reading the result: The indicators reveal which listed words occur, but not their order, tone or full meaning. A transparent baseline can offer available books sharing a user-selected subject tag. Compare a learned model against that baseline using an appropriate outcome and user choice. Additional sophistication should earn its place through useful evidence.

Try it yourself: Can a click label automatically be described as a satisfaction label?

Check your reasoning: No. It records a click. A reader may click and dislike the item, or enjoy an item without clicking in the measured interface.

9.3 Place the model inside a service

An AI model is usually one component in a larger system. The library must collect permitted inputs, transform them consistently, run the model, interpret its output and decide what action follows. A strong model can still produce a poor service if another component uses outdated information or hides uncertainty.

A practical pipeline has an input contract, preprocessing, inference, a decision policy and monitoring. Monitoring means watching relevant behavior after release, such as error counts or changes in incoming data. Distribution shift occurs when new cases differ from the cases used to

develop the model. A fallback is a defined alternative when the model cannot be used appropriately, such as ordinary search or human help.

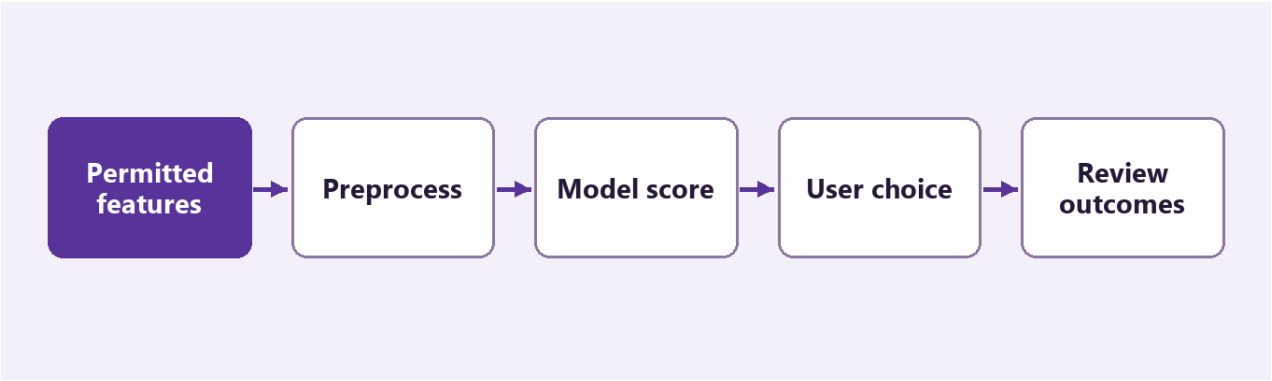


FIG-AI-09-03 The final action is a service decision, not merely a number. The user can decline suggestions while exact catalogue functions remain available.

Investigate and explain

A model ranks A above B, but A is already on loan. Describe a rule that prevents the recommendation interface from promising unavailable copies. Then define the response when required features are missing.

Condition	Service action	Evidence to retain
Candidate unavailable	Filter or mark unavailable	Current catalogue status
Features missing	Use baseline or ask appropriately	Missing-field reason
User declines	Skip personalization	Ordinary service continues
Unusual error pattern	Review and possibly pause model	Observed cases, not guesses

Reading the result: Filtering by availability uses a current fact that the preference model need not learn. Missing inputs should not silently become invented histories. Keep monitoring focused on necessary evidence and protect personal data. This division of responsibilities makes the service easier to test and easier to explain to the people who use it.

Try it yourself: Does a model score of 2.5 mean 2.5 minutes?

Check your reasoning: Only if the task, target units and output definition establish that interpretation. A raw score alone has no automatic physical units.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Design a scanned-book request pipeline. Separate image recognition, exact ID lookup, availability and optional recommendation. Give a recovery path for an uncertain recognition result.

Worked guidance

Recognition proposes an ID; catalogue lookup checks the ID; availability determines whether the copy may be held. A human can correct the scan or enter the ID directly. Recommendation remains optional and cannot replace those exact checks.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can translate images and text into defined inputs and place prediction within a checkable service. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

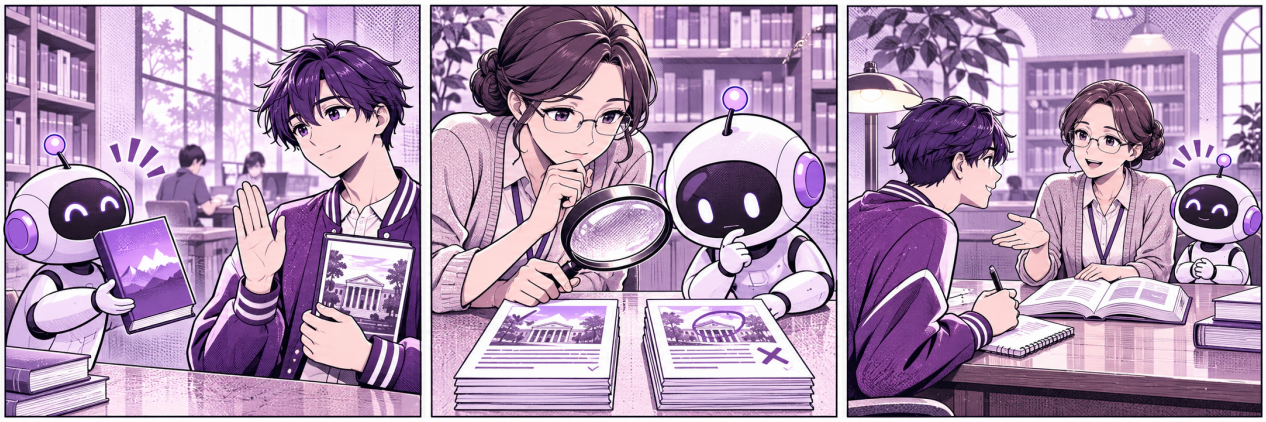
AI-CH10 / CHAPTER 10

Limitations, Ethics and Responsible AI

BY THE END

Explain uncertainty, privacy choices, group evaluation and effective human review.

Responsible design asks what the service does to people, what evidence supports it and how mistakes can be corrected.



ART-AI-CH10 Choice, evidence review and human support belong in the design of the service. A reader can decline a suggestion and still receive ordinary library help.

10.1 Make a recommendation a reader can decline

Pip offers a history book to a reader who liked two earlier history titles. A suggestion can be welcome, but borrowing records can reveal private interests. Start by asking what information the system actually needs and whether the reader has agreed to use it. Offer a useful catalogue service even when personalization is declined.

Biased training records can produce unequal errors across users, languages or contexts. Sparse feedback can make a model repeat familiar recommendations and learn little about overlooked books. Ask who was represented, how labels were collected, and what the user can do when an output is inappropriate.

FIG-AI-10-01 The permission check precedes using personal history. A separate exact-book lookup can operate on a book identifier. A correction or decline should not block that lookup.

Look closer: An audit can begin with a small disparity and a question. Avoid writing a causal explanation into the results column before collecting evidence. Preserve the denominator, the sampling conditions and the limits of the claim.

Audit a model before offering it

Audit two fictional groups of twenty labelled urgent messages each. Group A receives 18 correct urgent flags; group B receives 10. Predict what a pooled number conceals. Record a measurement, one plausible explanation to investigate, and one action that preserves users' ability to obtain help.

Group	True urgent	Correct flags	Misses
A	20	18	2
B	20	10	10
Combined	40	28	12

Reading the result: Pooled recall is 70%, but the group recalls are 90% and 50%. The gap warrants investigation of language coverage, labels and sampling. These small counts establish a disparity in this example, not its cause. Provide a direct human escalation route and gather appropriate evidence before changing a consequential policy.

Try it yourself: Which parts of the Smart Library can still run if the model is switched off?

Check your reasoning: Exact catalogue lookup, request ordering and legal route calculation all use explicit procedures and structures. In the capstone you will connect them and decide where an optional prediction earns its place.

10.2 Separate usefulness from permission

A reader might welcome personalized suggestions but dislike having every borrowed title retained indefinitely. Technical ability to collect a field does not settle whether that collection is appropriate. Begin with the service purpose and explain what the system needs to perform it.

For the classroom library design, personalization is optional. Exact lookup, borrowing and help remain available when a reader declines. Collect only the information needed for the stated purpose, define who can access it and plan when it should be removed. Permission does not make every use sensible; nor does a convenient dataset guarantee that its original purpose supports a new one. This is a design exercise rather than a statement of the law in any jurisdiction.

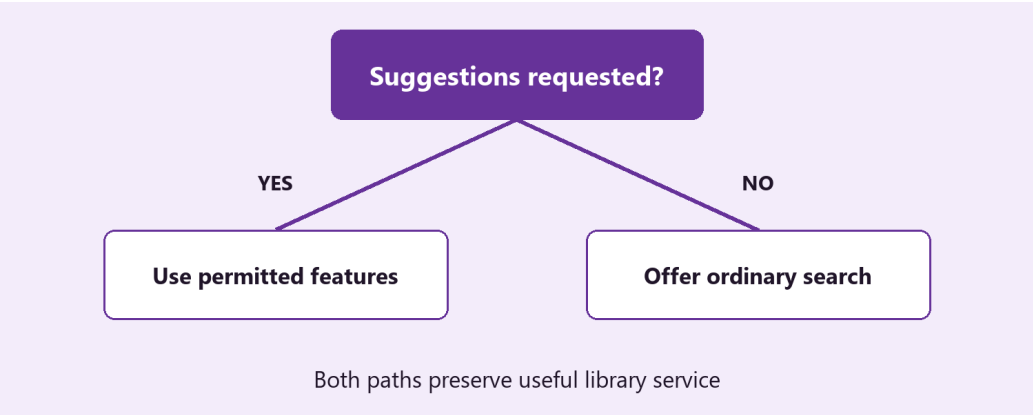


FIG-AI-10-02 Both branches lead to useful library service. A refusal is a choice about data use, not evidence that the reader dislikes every book.

Investigate and explain

Design the fields for an optional tag-based recommendation. Compare storing chosen subject tags for one session with storing complete borrowing history forever. Explain which choice is sufficient for your stated baseline.

Field	Needed for session tag match?	Reason
Selected subject tags	Yes	Define current interests
Available candidate tags	Yes	Allow matching
Complete borrowing history	No	Baseline does not use it
Sensitive personal profile	No	No justified role in this task

Reading the result: The minimal baseline compares current chosen tags with available candidate tags. It needs no historical profile. If a later model proposes additional fields, justify their use separately and

evaluate whether the improvement is worthwhile. Keeping a small, understandable purpose makes both explanation and testing easier.

Try it yourself: Should declining personalization become a negative preference label?

Check your reasoning: No. It records a choice about the feature, not a rating of a particular book.

10.3 A group gap is a question to investigate

A pooled score can hide different experiences. Suppose urgent-message recall is 18 out of 20 for group A and 10 out of 20 for group B. The overall fraction is 28 out of 40, but that one number does not describe either group well. Looking at groups can reveal a problem that an average conceals.

Group evaluation requires a justified purpose, careful handling of sensitive information and enough appropriate examples. Differences can reflect sampling, labels, missing features, deployment conditions or other factors. A gap alone does not identify a cause. Fairness also involves which errors matter and who can challenge them; there is no single score that settles every value judgment. Report counts alongside rates so readers can see the evidence behind a percentage.

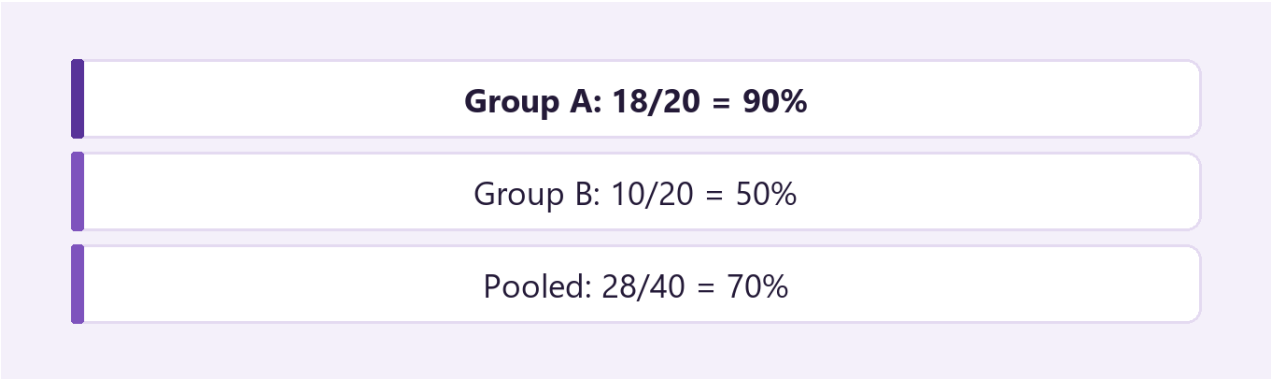


FIG-AI-10-03 The denominators are actual urgent messages in each group. These numbers describe recall only; they do not report false positives or overall accuracy.

Investigate and explain

Write a short review note for the manager. Calculate the three recalls, identify two missing kinds of evidence and propose a limited next evaluation. Avoid claiming a cause that the data do not show.

Question	Available evidence	Missing evidence
How many urgent cases detected?	18 and 10	Performance on larger representative samples
How many false alarms?	Not supplied	Routine-message predictions

Question	Available evidence	Missing evidence
Why the gap?	Not established	Data, label and context investigation

Reading the result: A defensible note says the observed recall differs and deserves investigation before relying on the system. It asks for false-alarm counts, representative held-out cases and a review of data quality. It can propose human review for uncertain or consequential cases while evidence improves. The decision should be tied to consequences and a clear acceptance criterion, not to a comforting pooled number.

Try it yourself: Can 70% pooled recall establish that each group has 70% recall?

Check your reasoning: No. Aggregation hides the group rates. Here they are 90% and 50%, even with equal group sizes.

10.4 Make a decision someone can challenge

A reader receives an unhelpful recommendation. Another reader cannot find a requested title because an image model misread the copy number. The service should provide a useful way to correct these outcomes. Showing a confident number is less useful than making the relevant evidence and next action understandable.

A concise model record can state the intended task, data source and permission basis, input definitions, evaluation setting, important limitations and a contact or review route. A human fallback needs an actual process: who receives a request, what information they see and how a correction returns to the user. Human review is not automatically effective if the person lacks time, authority or reliable evidence.

FIG-AI-10-04 The arrows show an accountable process. A correction can repair a record without being silently added as a training label until its quality and permitted use are checked.

Investigate and explain

Draft four sentences for a library recommendation card: purpose, uncertainty, choice and help. Then state a condition under which the team should pause the feature and investigate.

Element	Example wording
Purpose	Suggested because it shares your selected tags
Uncertainty	This may not match your interests
Choice	Skip suggestions and continue with ordinary search
Help	Ask the library desk to review a missing or incorrect result

Reading the result: A useful pause condition is concrete: for example, a reproducible error that exposes private history, or a sustained rise in verified harmful mistakes under a team-defined review policy. Avoid promising perfect prediction or meaningful explanations for every complex model. The record should describe evidence the team actually possesses, with unresolved questions made visible.

Try it yourself: What makes a human fallback more than a decorative button?

Check your reasoning: An identified responsible person or service, access to the necessary evidence, authority to correct the outcome, and a usable response process.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Write a one-page model record for optional library suggestions. Include purpose, permitted inputs, baseline, evaluation population, errors to inspect, user choice and a named role responsible for review. Mark evidence not yet collected.

Worked guidance

A defensible record separates supplied toy arithmetic from actual trained performance. It does not invent test results or reviewer approval. It preserves ordinary service for users who decline, describes necessary data and retention, and defines how a librarian can correct or pause the feature when observed problems warrant investigation.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can explain uncertainty, privacy choices, group evaluation and effective human review. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

Smart Campus Library: integrated project

FIG-CAP-00 Connect exact records, service order, legal routes and optional predictions. This illustration is a story map; the following data define the reference solution.

Build the blueprint for a Smart Campus Library

A visitor asks for copy 24 to be delivered to Gate. Behind them another visitor wants copy 12. Pip has a catalogue, a service window and a walking map. Your job is to design the complete trip from the first request to a confirmed delivery. A useful design also explains what happens when a copy is missing or a walkway closes.

Prepare your desk

Use a blank sheet for the system, four location cards, three catalogue records and two request slips. Copy the data on the next page. Give each request its own number. You can complete the project with pencil and paper. The reference-design sections below form a worked reference solution; cover them on the first attempt.

What to hand in

Draw the stored records and label each operation. Trace both requests, including the order in which they start service. On the map mark the least-time route for the first request, show the arithmetic and repeat after a closure. Add one optional book suggestion and an equally usable path for a reader who declines it.

What counts as success

A second reader can tell which copy moved, which request was served first, and why each movement was legal. They can explain one place where prediction might help and one place where an exact fact must be checked. Leave enough evidence for that reader to reproduce the outcome.

The library's starting data

Use a sorted array of distinct copy IDs. A copy ID is a lasting identifier; a title is a display label that more than one copy could share. Each tuple, or group of fields, describes one physical copy. All held copies are collected at the Library before delivery on this small map.

Copy ID	Title	Shelf	Initial state
12	Ink Atlas	A1	Available
24	River Walks	B4	Available
37	Quiet Maps	C2	On loan

The request slips

R1 asks for copy 24 delivered to Gate and declines personalization. R2 asks for copy 12 at the Library counter. Admit R1, then R2. A single clerk starts requests in arrival order. Each request has its own status so a visitor can distinguish waiting, held, delivered and failed.

The map and the optional model

Use the two-way walkways L-C 2, C-B 3, L-B 9 and B-G 2 minutes. L is Library, C Café, B Lab and G Gate. The optional score uses two consented features, shared tags and prior likes. The demonstration weights are 0.5 and 0.2 with bias 0.1 and ReLU. They are supplied for tracing; this exercise contains no preference-training dataset.

Make a prediction

Before continuing, write down the first index inspected for copy 24, the next waiting request after R1 starts, and the time of the least-time walk from L to G. Predict whether declining personal recommendations will affect that walk.

Reference design from slip to receipt

A request enters the waiting queue only after the desk has checked that its identifier and destination fields are present. The clerk removes the front slip and searches the sorted catalogue by copy ID. It checks availability before marking a copy held for that specific request.

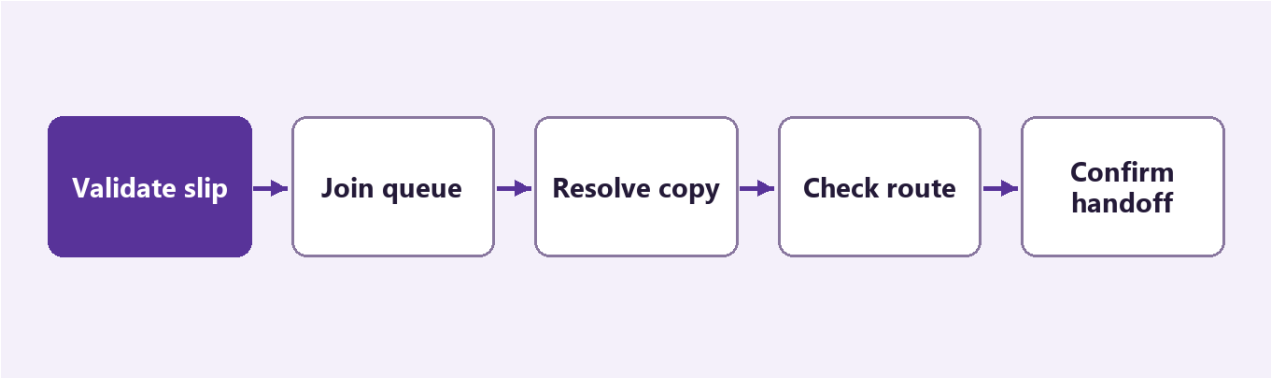


FIG-CAP-01 Each arrow passes a request or a result. An availability failure exits after lookup. An unreachable route releases the hold under this single-clerk policy. Confirmation requires an actual handoff.

Connect the parts explicitly

The queue stores request IDs in arrival order. An indexed array connects each copy ID with its shelf and availability. A weighted graph supplies legal walks. A separate request record carries status between these jobs. The queue need not contain the whole catalogue, and the graph need not know any book titles.

The optional branch

Offer a related title only after permission and only among eligible candidates. Keep this branch separate from the delivery decision. If a reader declines or the model is unavailable, the exact lookup and route can still finish. The receipt records what was delivered and where, independently of any guessed preference.

Trace the two requests

Follow R1 using the starting data in this project. Bounds 0 to 2 place the middle at index 1. That record has ID 24, so the clerk finds River Walks on shelf B4 in one inspection. Before collecting it, mark the available copy held for R1. Another request should see that hold.

Event	Waiting queue	Copy or request consequence
Admit R1 then R2	R1, R2	Both waiting
Start R1	R2	Copy 24 held for R1
Deliver R1 at G	R2	Copy 24 on loan, R1 delivered
Start R2	Empty	Copy 12 held for R2
Counter handoff	Empty	Copy 12 on loan, R2 delivered

Follow the second lookup

For R2 the first comparison is 12 against 24. Move high to 0. The next comparison finds copy 12 at index 0. The Library counter is the starting location, so a pickup there needs zero walking edges. Confirming the handoff is still necessary.

Change the starting condition

Reset the catalogue and make copy 24 already on loan. R1 must receive an unavailable response, and the clerk can then begin R2. A successful search answers “which record?”; availability answers “may this request use this copy?” They require separate checks.

A route you can defend

Start at L with the parcel and compare the simple routes to Gate. Gate has only one neighbour, so B-G must appear at the end. There are just two simple ways to reach B from L on this four-building map. Adding a positive-time cycle cannot improve either journey.

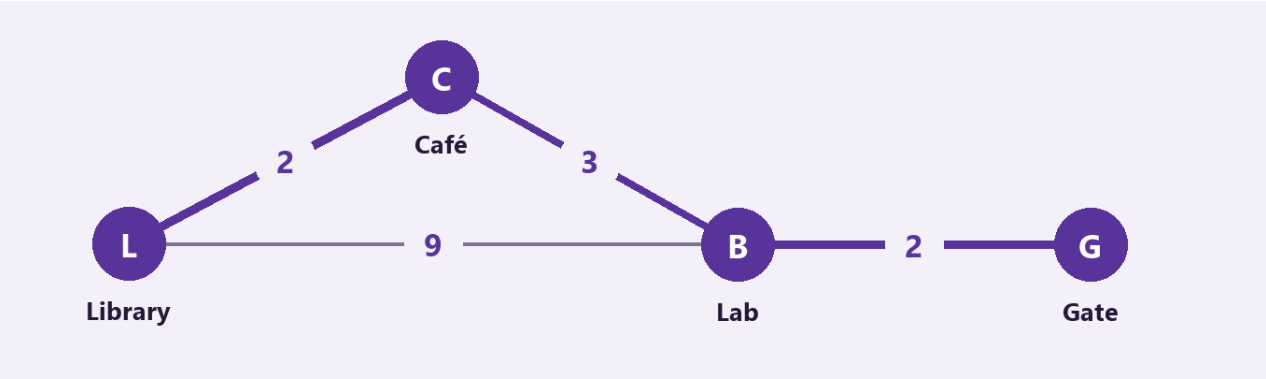


FIG-CAP-02 The direct L-B walkway takes nine minutes. Pip can reach B in five via C. The final two-minute leg to G is shared by both choices.

Walk	Legs	Travel time
L-B-G	2	9 + 2 = 11 minutes
L-C-B-G	3	2 + 3 + 2 = 7 minutes

A closure after the order is placed

Close C-B before departure. Recompute on the current map: the three-leg option is illegal, so use the eleven-minute walk. If B-G also closes, G is unreachable. Report the failure and release the hold if the parcel has not left the Library. A different recovery policy after departure would require knowing the parcel's actual location.

An optional recommendation

Imagine a third reader explicitly requests a suggestion. Candidate A shares two tags with their request and has three prior likes for its author. Candidate B shares three tags and has no prior likes. Both books are available. Which would the demonstration model rank higher?

Candidate	Tag term	Like term	Bias	ReLU score
A	2 × 0.5 = 1	3 × 0.2 = 0.6	0.1	1.7
B	3 × 0.5 = 1.5	0 × 0.2 = 0	0.1	1.6

Interpret the small difference

The numerical rule ranks A first by 0.1. A small margin does not establish a real preference. These weights were supplied by the author, and the arithmetic alone cannot show how frequently a real reader would agree. One transparent alternative is to present available candidates grouped by the subject the visitor selected.

A fair test of usefulness

With permission, collect examples of later voluntary feedback. Keep related reader records together when splitting data, to avoid measuring recognition of the same person as if it were performance on a new reader. Compare the trained method against a simple tag rule on held-out cases. Look for errors across relevant groups and allow a reader to turn suggestions off.

Break the design on purpose

A good test has a known starting state, a concrete action and an observable result. Begin each row from a fresh copy of the starting data unless the row explicitly gives a different state. Ask a partner to trace the design, then compare its outputs and stored changes with these expectations.

Change	Expected result	State to inspect
Request absent ID 99	Missing copy	No hold created
Request on-loan ID 37	Unavailable	Copy remains on loan
Submit R1 then a second request for 24	Second cannot claim same copy	First hold or loan visible
Close B-G before departure	No route to G	R1 fails, hold released
Decline suggestions	Ordinary delivery succeeds	No history read
Request counter pickup	Zero walking time	Handoff still confirmed

Diagnose a failure

Suppose the receipt says “delivered” even though Gate is isolated. Locate the first wrong transition. If the route calculation reported unreachable, the error lies in handing a failure forward as a success. If it reported a path through B-G, the route calculation probably saw an obsolete map.

Finish your blueprint

Write one sentence per component stating its input and output. Circle any sentence that says only “the AI handles it.” Replace it with a checkable rule, a stored record or a prediction whose limitations you can describe. You now have a system another learner can inspect and test.

Review across the three strands

A search and a promise

A catalogue is sorted by title. The clerk seeks copy ID 37 using midpoint comparisons on IDs. Predict the problem and repair it two ways. A title order need not put IDs in increasing order. Scan the existing row by ID, or maintain an additional ID-sorted index and use binary search on that index.

A process and its evidence

Two visitors arrive A, then B. A history entry X is saved during service. When Undo removes B from the waiting line, which assumption failed? Request order and edit order have been merged. A queue should retain B as the next waiting visitor; a separate stack should remove X when the edit is undone.

A score and its limits

Inputs 2 and 3, weights 0.5 and 0.2 and bias -2 give a negative sum. Calculate it and predict ReLU. The sum is 1 + 0.6 - 2 = -0.4, giving output zero. Changing the bias to -1.9 changes the sum to -0.3 but leaves ReLU at zero. The same final output can conceal different internal scores.

Return to the first page of questions

Retrieve your notes from the beginning of your study. Pick one answer you can now justify more precisely. Give the argument, a counterexample to a weaker rule, and a trace another reader can repeat. Then open the separate Test Bank for an unfamiliar case.

Project rubric | 24 marks: Score six areas, each 0-4: identity and state, search, service order, routes, optional AI with user choice, and failure tests. Use 0=no evidence; 1=partial; 2=mostly correct with a gap; 3=correct and traceable; 4=justified and boundary-tested. This is a peer-review rubric, not an instructor-issued grading scheme.

Glossary and retrieval cues

Accuracy: The share of predictions whose classes match the labels. An always-clear scanner can be accurate overall while missing blurred pages. [AI-CH07/08/10]

Activation: A rule applied to a unit's weighted sum. ReLU maps -0.4 to zero. [AI-CH02/03/04]

Architecture: The arrangement of units and connections in a model. A two-input, two-hidden, one-output network is one architecture. [AI-CH05/06]

Backpropagation: A calculation that works backward through a network to find how loss responds to its parameters. The optimizer uses the result. [AI-CH07/08/10]

Bias: In a neuron, a number added once to its weighted sum. In an evaluation discussion, bias can refer to systematic distortions in data or outcomes. [AI-CH02/03/04, AI-CH07/08/10]

Classification: Prediction of a category, such as routine or urgent. A threshold can turn a numerical score into a class. [AI-CH05/06]

Epoch: One pass through the training examples. There may be several parameter updates within one epoch. [AI-CH07/08/10]

Feature: A property supplied as model input. A count of shared tags is one numerical feature. [AI-CH02/03/04]

Forward propagation: Computing a prediction from inputs through successive network units with given parameters. [AI-CH05/06]

Generalization: A model's ability to perform its task on relevant new examples. [AI-CH07/08/10]

Gradient: Numbers describing local sensitivity of loss to changes in parameters. They help an optimizer choose an update. [AI-CH07/08/10]

Hidden layer: A layer of intermediate computing units between the inputs and outputs. [AI-CH05/06]

Label: An observed target for a training or evaluation case. For a blurred-page detector it can be a human judgement of image quality. [AI-CH02/03/04, AI-CH07/08/10]

Loss: A numerical measure of discrepancy between prediction and target. Squared error is one possible loss. [AI-CH07/08/10]

Model: A parameterized mapping from described inputs to an output. A trained model has parameters adjusted from examples. [AI-CH02/03/04]

Neuron: A mathematical unit that combines weighted inputs and a bias, then applies an activation. [AI-CH02/03/04]

Node: An individual element of a list, tree, graph or network. Its fields and connections depend on the structure. [DS-CH03, DS-CH06, AI-CH05/06]

Optimizer: A procedure that adjusts model parameters to improve a training objective. It can use gradients. [AI-CH07/08/10]

Overfitting: Learning details of training cases in ways that harm performance on new cases. Diverging learning curves can be a clue. [AI-CH07/08/10]

Parameter: An adjustable model number such as a weight or bias. A forward pass reads it; training can change it. [AI-CH02/03/04, AI-CH07/08/10]

Prediction: An output a model produces for an input. Its accuracy must be evaluated against relevant outcomes. [AI-CH02/03/04, AI-CH07/08/10]

Recall: The fraction of actual positive cases detected. Six recognized blurred scans out of ten give 60% recall. [AI-CH07/08/10]

Regression: Predicting a numerical quantity, such as waiting time in minutes. [AI-CH05/06]

ReLU: The rectified linear unit activation. It returns zero below zero and returns its input for nonnegative values. [AI-CH02/03/04]

Sigmoid: A smooth activation that maps a real input to a value strictly between zero and one. Zero maps to one half. [AI-CH02/03/04]

Step activation: A threshold rule with two outputs. This book uses one when $z \geq 0$ and zero otherwise. [AI-CH02/03/04]

Test set: Held-out cases reserved for evaluation after design choices have been fixed. [AI-CH07/08/10]

Training: Using examples and a learning procedure to adjust model parameters toward an objective. [AI-CH02/03/04, AI-CH07/08/10]

Validation set: Examples used to guide model choices such as stopping time. Separate final test examples are still needed. [AI-CH07/08/10]

Vertex and weight: A vertex is a graph node. A graph edge weight can be a travel cost; a neural weight multiplies an input contribution. [DS-CH07, AI-CH02/03/04]

Batch: Examples processed together for an update. Twelve cases in groups of four make three batches. [AI-CH07]

Epoch: One pass through the chosen training dataset under the stated sampling convention. [AI-CH07]

Precision: True positives divided by predicted positives. The fraction is undefined if there are no predicted positives. [AI-CH08]

One-hot encoding: Indicators for category membership, such as zone B=[0, 1, 0] in order[A, B, C]. [AI-CH03]

Distribution shift: A difference between development cases and cases encountered later. [AI-CH09]

Embedding: A learned numerical vector used to represent an item or token. Optional preview. [AI-CH09]

Reading, sources and credits

The chapter structure follows the supplied three-volume Visual Edition V1: seven Algorithms chapters, eight Data Structures chapters and ten Connectionist AI chapters. The earlier project manuscript supplies revised stories and worked cases. This edition adds separate teaching sequences, new practice and a larger assessment companion. The page limit was relaxed by the user to allow teaching depth and visuals.

But what is a Neural Network?

Grant Sanderson / 3Blue1Brown | AI-CH02 to AI-CH06

The lesson introduces a network through handwritten-digit recognition.

https://www.3blue1brown.com/lessons/neural-networks/

Machine Learning Crash Course

Google for Developers | AI-CH05 to AI-CH10

The official course has focused modules for neural networks, classification, data splits and generalization.

https://developers.google.com/machine-learning/crash-course

TensorFlow Playground

TensorFlow Playground project | AI-CH03 to AI-CH08

A browser neural-network sandbox makes changing features, layers and training settings visible.

https://playground.tensorflow.org/

Learn the Basics

PyTorch documentation contributors | After AI-CH05 to AI-CH08

The official tutorial organizes a machine-learning workflow around data, model construction, automatic differentiation and optimization.

https://docs.pytorch.org/tutorials/beginner/basics/intro.html

Web review date: 6 November 2026. Public page review is distinguished from interactive execution in the Learning Materials companion. No external images or large source passages are reproduced. Technical diagrams are original project drawings; the anime illustrations were generated with the built-in image tool using the supplied visual direction. AI assistance supported writing, assessment, art and production.

Project evidence includes earlier source-review and verification records. The approved Markdown Content V2.0, a separate grading rubric and edition-specific instructor feedback were unavailable in that record and remain items for the team to reconcile. This publication does not claim instructor approval, independent student testing or a fresh full viewing of the existing video library. Editable source files and the verification report accompany the PDFs.